Linear Discriminant Analysis (LDA) and Logistic Regression

LAB 7 and 8

- *Created by: Angeline A*
- *Reg No: 2348409*
- *3-MDS 'B'*
- Submitted on: 05/04/2024

In the below code, we are importing pandas library which is used to do data manipulation and analysis.

```
import pandas as pd
```

In the below code, we are importing the data and storing it in a variable named dataframe.

```
df = pd.read_csv('/content/data.csv')
df
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	conca
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	
...	...	...	...	...	...	...	...	...	...	
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	

569 rows × 32 columns

The below code snippet provides a concise summary of the dataset, including the number of samples, number of features, and data types of each column. It's useful for getting a quick understanding of the dataset's structure and characteristics.

```
# Display basic information about the dataset
print("Basic information about the dataset:")
print("-----------------------------------")
print("Number of samples:", df.shape[0])  # Number of rows, each row represents a sample
print("Number of features:", df.shape[1] - 2)  # Number of columns, excluding 'diagnosis' and 'id' columns
print("Data types:")
print(df.dtypes)
```

```
Basic information about the dataset:
-----------------------------------
Number of samples: 569
Number of features: 30
Data types:
id                           int64
diagnosis                   object
radius_mean                float64
texture_mean               float64
perimeter_mean             float64
area_mean                  float64
smoothness_mean            float64
compactness_mean           float64
concavity_mean             float64
concave_points_mean        float64
symmetry_mean              float64
fractal_dimension_mean     float64
radius_se                  float64
texture_se                 float64
```

```
perimeter_se               float64
area_se                    float64
smoothness_se              float64
compactness_se             float64
concavity_se               float64
concave_points_se          float64
symmetry_se                float64
fractal_dimension_se       float64
radius_worst               float64
texture_worst              float64
perimeter_worst            float64
area_worst                 float64
smoothness_worst           float64
compactness_worst          float64
concavity_worst            float64
concave_points_worst       float64
symmetry_worst             float64
fractal_dimension_worst    float64
dtype: object
```

- The above output provides basic information about the Breast Cancer Dataset:
  - Number of samples: There are 569 samples (rows) in the dataset. Each sample represents a patient or observation.
  - Number of features: There are 30 features (columns) in the dataset. These features represent various measurements and characteristics of breast cancer tumors, such as radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension.
  - Data types: The 'id' column is of type int64, and the 'diagnosis' column is of type object. The 'diagnosis' column likely contains categorical data with two categories ('M' for malignant and 'B' for benign). The remaining columns (radius_mean to fractal_dimension_worst) are of type float64, indicating they contain numerical data representing tumor characteristics.

In the below code, we are printing the first five rows of the dataset to understand the data.

```
# Display the first few rows
print("First few rows of the dataset:")
print("-----------------------------------")
df.head()
```

```
First few rows of the dataset:
-----------------------------------
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	

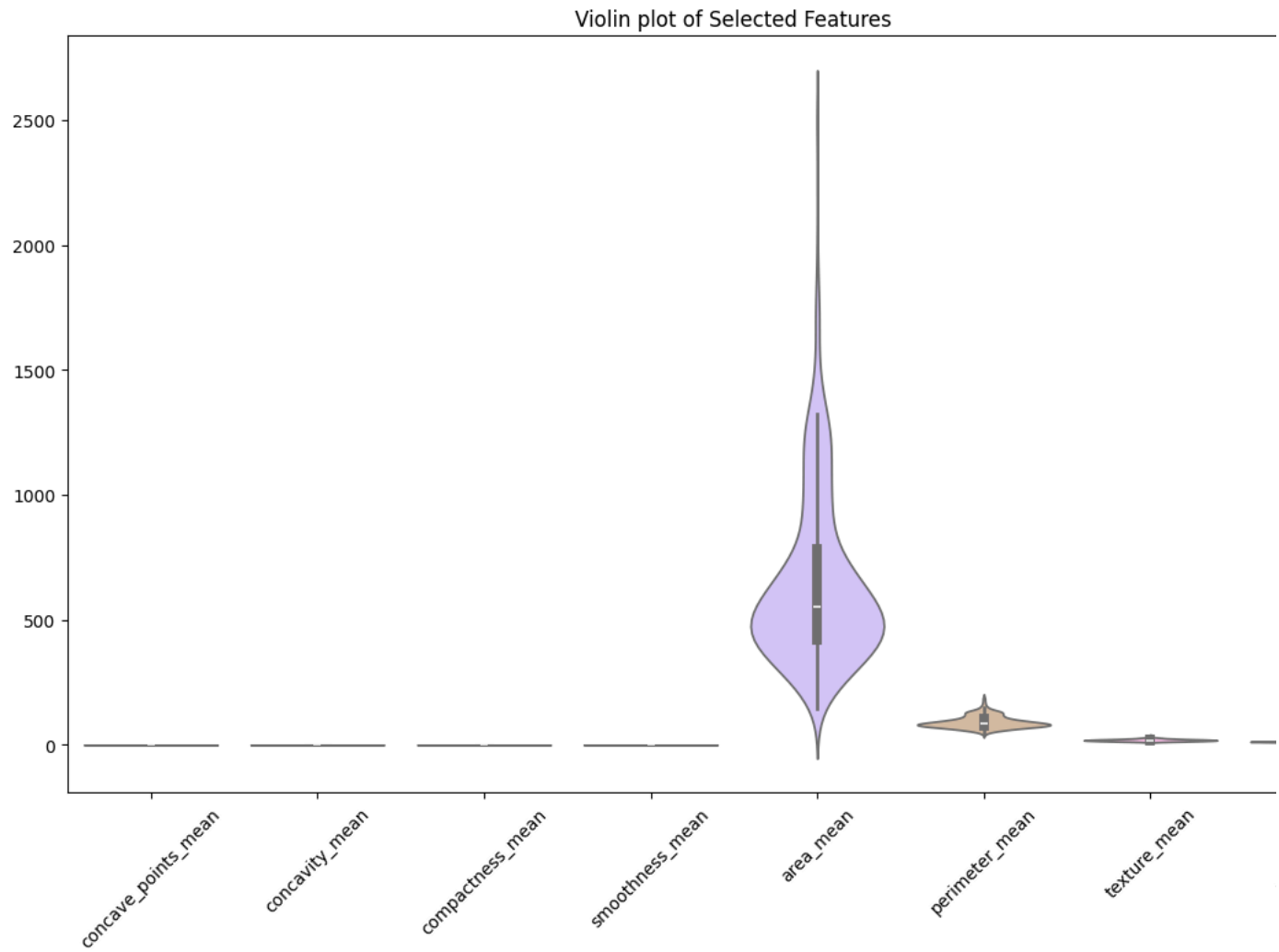
5 rows × 32 columns

In the below code we are importing the necessary libraries for visualizations.

```
import matplotlib.pyplot as plt
import seaborn as sns

```

In the below code, we are generating a violin plot to understand the distribution of the dataset.

```
# Box plot for selected features
selected_features = ['concave_points_mean', 'concavity_mean', 'compactness_mean',
                     'smoothness_mean', 'area_mean', 'perimeter_mean',
                     'texture_mean', 'radius_mean']
# Violin plot for selected features
plt.figure(figsize=(14, 8))
sns.violinplot(data=df[selected_features], palette='pastel')
plt.title('Violin plot of Selected Features')
plt.xticks(rotation=45)
plt.show()
```

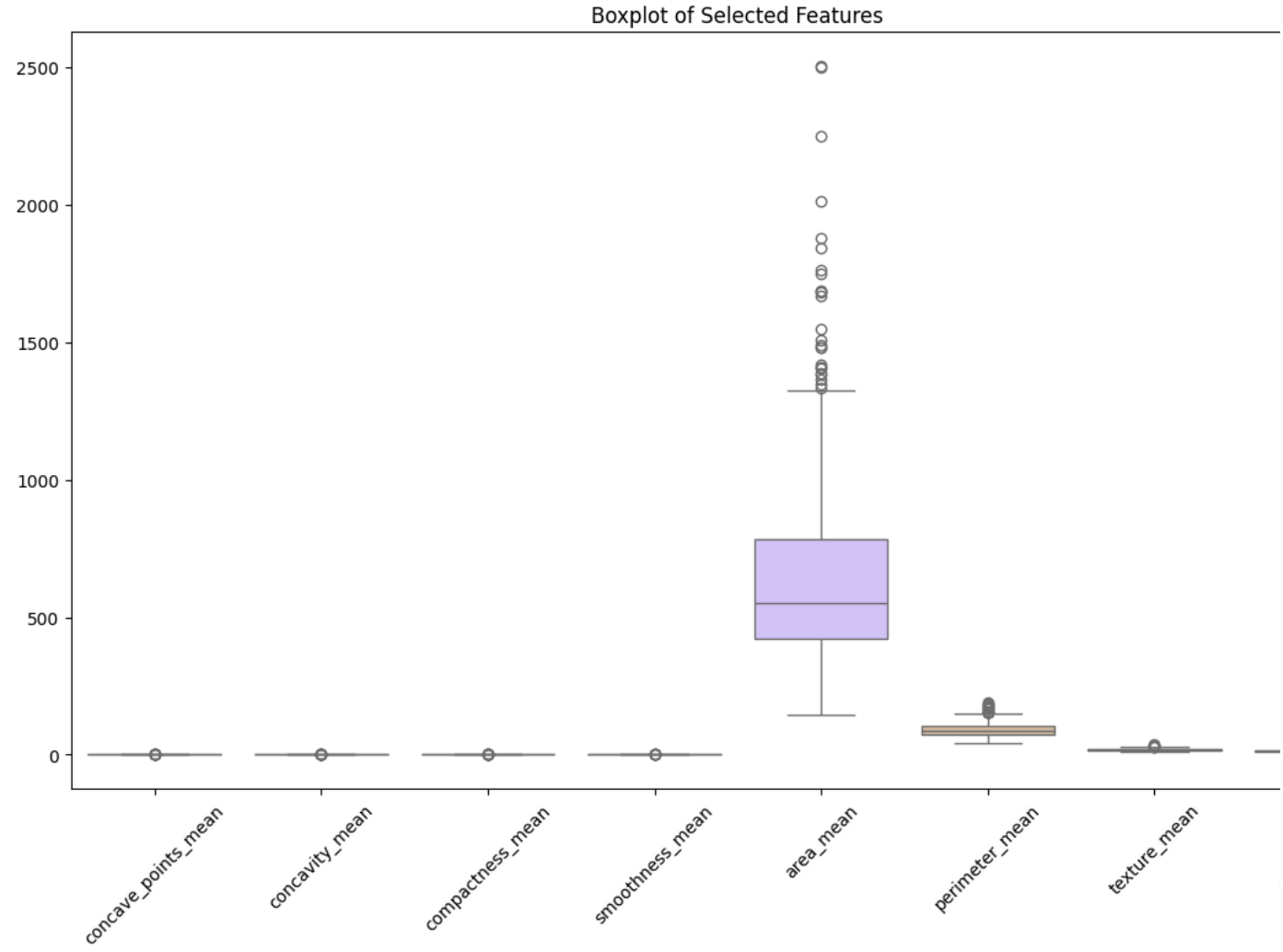


From the above graph we can infer the following,

- In this specific violin plot, the features on the x-axis are “concave points mean”, “concavity mean”, “compactness mean”, “smoothness mean”, “area mean”, “perimeter mean”, “texture mean”, and “radius mean”. The y-axis shows the number of features.
- It appears that the data for most features is clustered around the median, with a few outliers on either side.

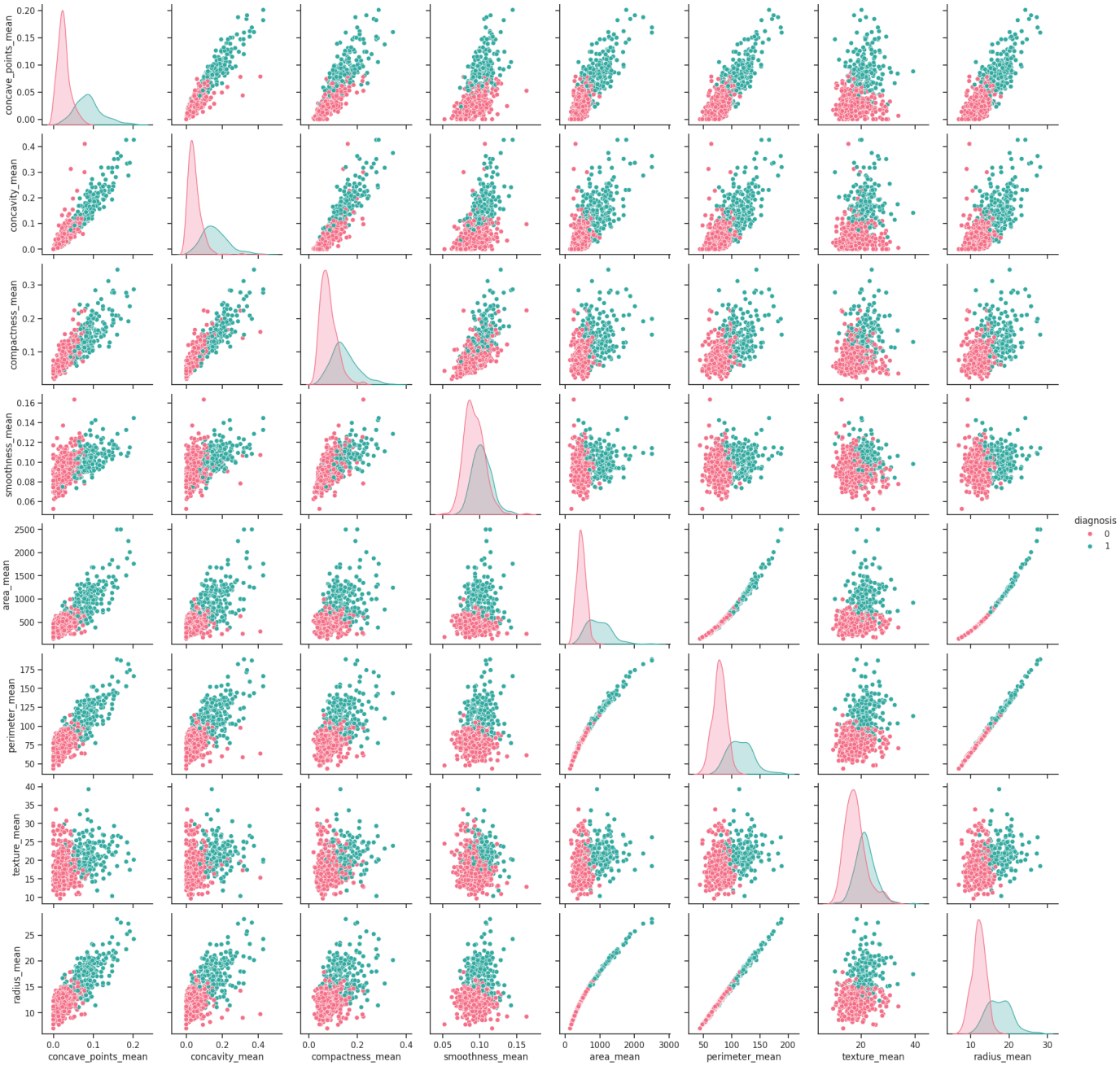
The below code generate a box plot which is a useful visualization for understanding the distribution, spread, and potential outliers within each selected feature in the dataset.

```
plt.figure(figsize=(14, 8))
sns.boxplot(data=df[selected_features], palette='pastel')
plt.title('Boxplot of Selected Features')
plt.xticks(rotation=45)
plt.show()
```



The below code generates the pair plot that allows for the visualization of pairwise relationships between the selected numeric features and the diagnosis column, providing insights into the distribution and relationships of these variables.

```
# Convert 'diagnosis' column to integer values
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Selecting the numeric features for pair plot
selected_features = ['concave_points_mean', 'concavity_mean', 'compactness_mean',
                     'smoothness_mean', 'area_mean', 'perimeter_mean',
                     'texture_mean', 'radius_mean']

# Adding 'diagnosis' to selected features for hue
selected_features_with_diagnosis = selected_features + ['diagnosis']

# Set the style for seaborn
sns.set(style="ticks", color_codes=True)

# Create pair plot
sns.pairplot(df[selected_features_with_diagnosis], hue='diagnosis', palette='husl', diag_kind='kde')
plt.show()
```



From the above we can infer many information, the few information are mentioned below,

- Observation 1: The data points show a positive diagonal cluster. This suggests a positive correlation, where higher values in "concave_points_mean" tend to be accompanied by higher values in "concavity_mean".
- Observation 2: The KDE plots for both diagnoses on the diagonal show some overlap but also a slight separation. This might indicate a generally similar distribution of values for both diagnoses, but with a possibility of some "concavity_mean" values being higher for diagnosis 1 compared to diagnosis 0.

The below code provides a summary of basic descriptive statistics for the numerical columns in the dataset, offering insights into the central tendency, variability, and distribution of the data.

```
# Select numerical columns for analysis (excluding 'diagnosis' and 'id')
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Calculate basic descriptive statistics
basic_stats = df[numerical_cols].describe().transpose()

# Display basic descriptive statistics
print("Basic Descriptive Statistics:")
print(basic_stats)
```

```
concavity_mean                0.029560       0.061540  1.307000e-01
concave_points_mean           0.020310       0.033500  7.400000e-02
symmetry_mean                 0.161900       0.179200  1.957000e-01
fractal_dimension_mean        0.057700       0.061540  6.612000e-02
radius_se                     0.232400       0.324200  4.789000e-01
texture_se                    0.833900       1.108000  1.474000e+00
perimeter_se                  1.606000       2.287000  3.357000e+00
area_se                      17.850000      24.530000  4.519000e+01
smoothness_se                 0.005169       0.006380  8.146000e-03
compactness_se                0.013080       0.020450  3.245000e-02
concavity_se                  0.015090       0.025890  4.205000e-02
concave_points_se             0.007638       0.010930  1.471000e-02
symmetry_se                   0.015160       0.018730  2.348000e-02
fractal_dimension_se          0.002248       0.003187  4.558000e-03
radius_worst                 13.010000      14.970000  1.879000e+01
texture_worst                21.080000      25.410000  2.972000e+01
perimeter_worst              84.110000      97.660000  1.254000e+02
area_worst                  515.300000     686.500000  1.084000e+03
smoothness_worst              0.116600       0.131300  1.460000e-01
compactness_worst             0.147200       0.211900  3.391000e-01
concavity_worst               0.114500       0.226700  3.829000e-01
concave_points_worst          0.064930       0.099930  1.614000e-01
symmetry_worst                0.250400       0.282200  3.179000e-01
fractal_dimension_worst       0.071460       0.080040  9.208000e-02

                                   max
id                        9.113205e+08
diagnosis                 1.000000e+00
radius_mean               2.811000e+01
texture_mean              3.928000e+01
perimeter_mean            1.885000e+02
area_mean                 2.501000e+03
smoothness_mean           1.634000e-01
compactness_mean          3.454000e-01
concavity_mean            4.268000e-01
concave_points_mean       2.012000e-01
symmetry_mean             3.040000e-01
fractal_dimension_mean    9.744000e-02
radius_se                 2.873000e+00
texture_se                4.885000e+00
perimeter_se              2.198000e+01
area_se                   5.422000e+02
smoothness_se             3.113000e-02
compactness_se            1.354000e-01
concavity_se              3.960000e-01
concave_points_se         5.279000e-02
symmetry_se               7.895000e-02
fractal_dimension_se      2.984000e-02
radius_worst              3.604000e+01
texture_worst             4.954000e+01
perimeter_worst           2.512000e+02
area_worst                4.254000e+03
smoothness_worst          2.226000e-01
compactness_worst         1.058000e+00
concavity_worst           1.252000e+00
concave_points_worst      2.910000e-01
symmetry_worst            6.638000e-01
fractal_dimension_worst   2.075000e-01
```

From the above result, we can observe the following for each of the columns,

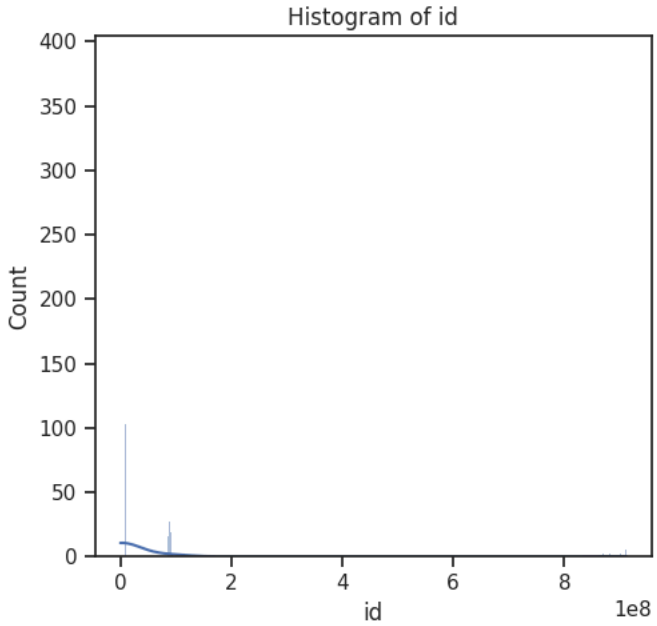
- id:
  - This column represents unique identifiers for each observation.
  - It has a count of 569, indicating that there are no missing values.
  - The minimum ID is 8670, and the maximum ID is 9113205.
- diagnosis:
  - This column contains categorical values indicating the diagnosis of tumors ('M' for malignant and 'B' for benign).
  - The count is 569, indicating no missing values.

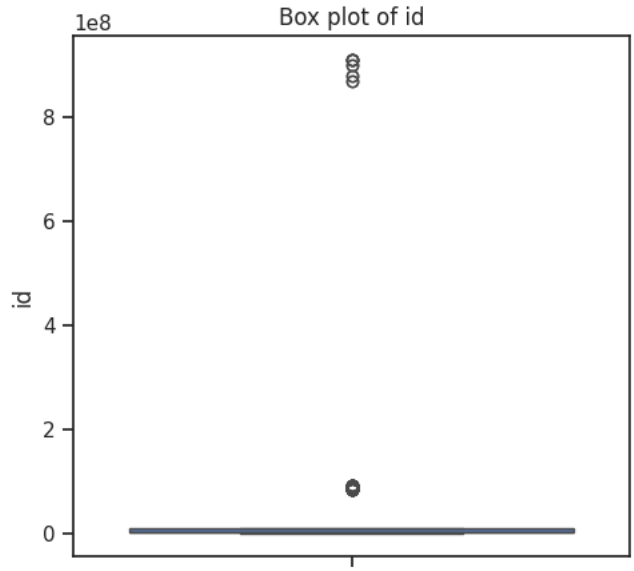
    - The mean is approximately 0.372, suggesting that about 37.2% of tumors are malignant on average.

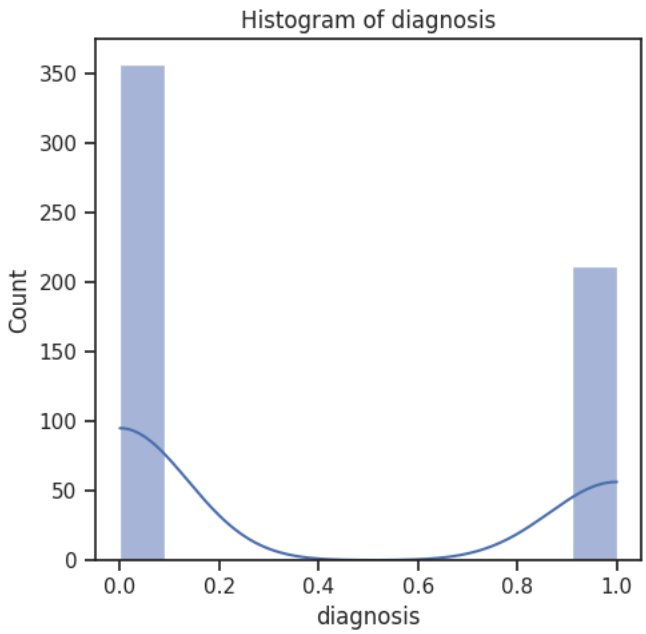
- radius_mean:
    - This column represents the mean radius of tumors.
    - The mean radius is approximately 14.13 units, with a standard deviation of approximately 3.52 units.
    - The minimum radius is 6.98 units, and the maximum radius is 28.11 units.
- texture_mean:
    - The mean texture of tumors is approximately 19.29 units, with a standard deviation of approximately 4.30 units.
    - The minimum texture is 9.71 units, and the maximum texture is 39.28 units.
- perimeter_mean:
    - The mean perimeter of tumors is approximately 91.97 units, with a standard deviation of approximately 24.30 units.
    - The minimum perimeter is 43.79 units, and the maximum perimeter is 188.50 units.
- area_mean:
    - The mean area of tumors is approximately 654.89 square units, with a standard deviation of approximately 351.91 square units.
    - The minimum area is 143.50 square units, and the maximum area is 2501.00 square units.
- smoothness_mean:
    - The mean smoothness of tumors is approximately 0.096, with a standard deviation of approximately 0.014.
    - The minimum smoothness is 0.05263, and the maximum smoothness is 0.1634.
- compactness_mean:
    - The mean compactness of tumors is approximately 0.104, with a standard deviation of approximately 0.053.
    - The minimum compactness is 0.01938, and the maximum compactness is 0.3454.
- concavity_mean:
    - The mean concavity of tumors is approximately 0.089, with a standard deviation of approximately 0.080.
    - The minimum concavity is 0, and the maximum concavity is 0.4268.
- concave_points_mean:
    - The mean number of concave points of tumors is approximately 0.049, with a standard deviation of approximately 0.039.
    - The minimum number of concave points is 0, and the maximum number of concave points is 0.2012.

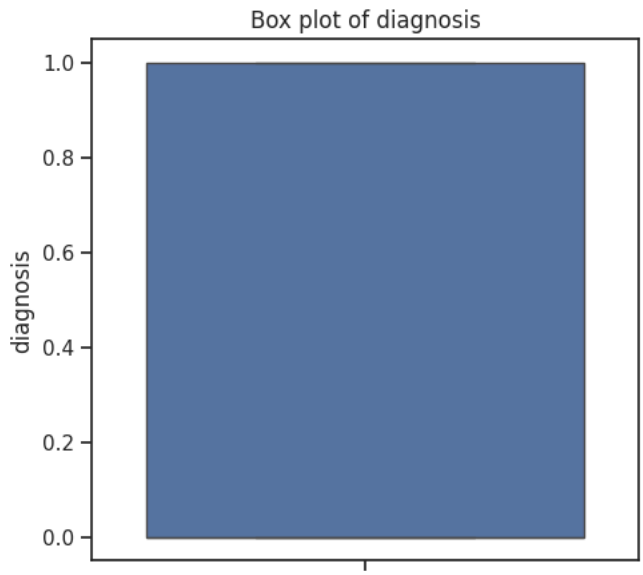
The below code efficiently creates paired histograms and box plots for each numerical column, allowing for visual inspection of the distribution and variability of each feature in the dataset.

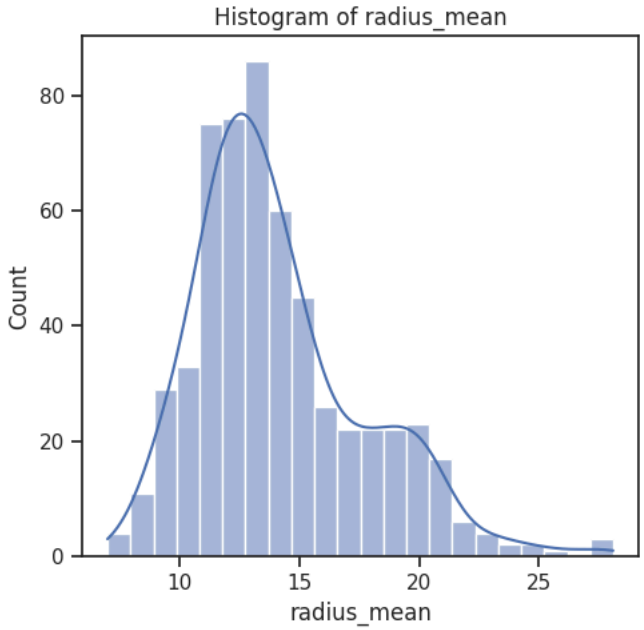
```
# Visualize the distribution using histograms and box plots
for col in numerical_cols:
    plt.figure(figsize=(10, 5))

    # Histogram
    plt.subplot(1, 2, 1)
    sns.histplot(data=df, x=col, kde=True)
    plt.title(f'Histogram of {col}')

    # Box plot
    plt.subplot(1, 2, 2)
    sns.boxplot(data=df, y=col)
    plt.title(f'Box plot of {col}')

    plt.tight_layout()
    plt.show()
```

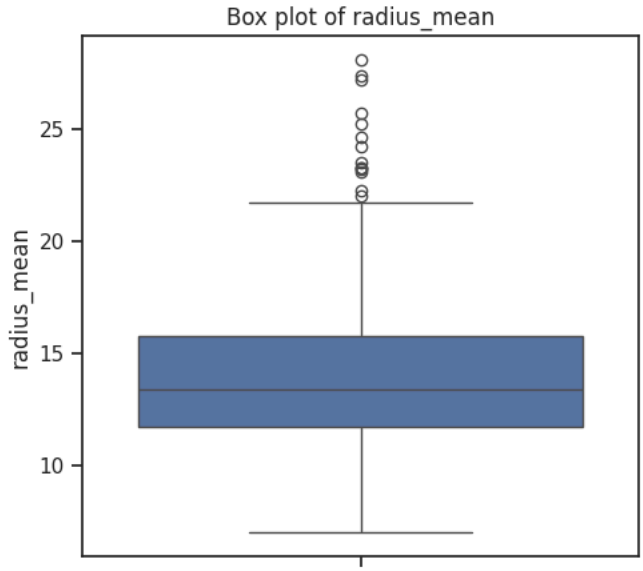

Histogram of id
Count
id
1e8

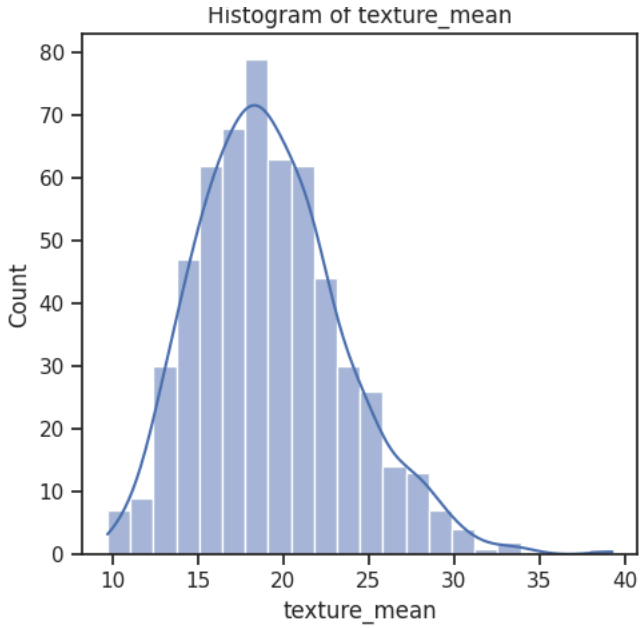

Box plot of id
1e8
id

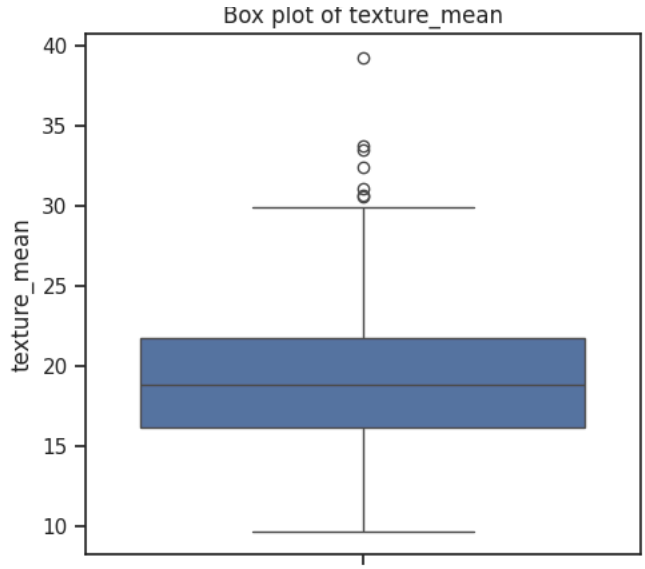

Histogram of diagnosis
Count
diagnosis

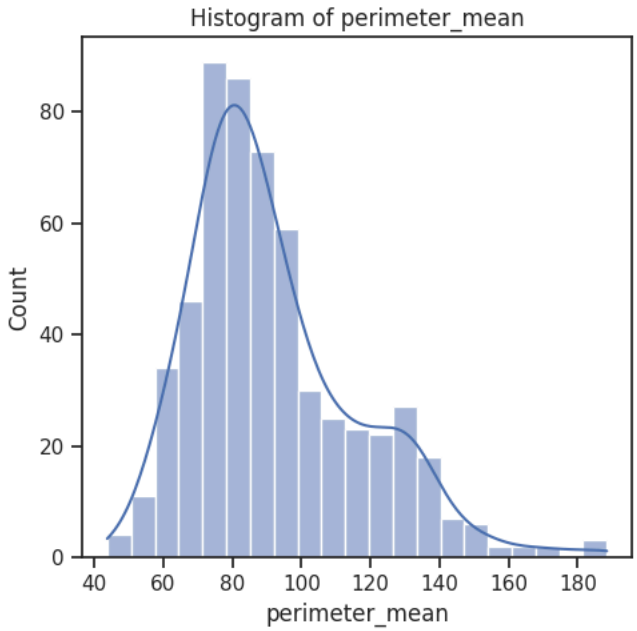

Box plot of diagnosis
diagnosis

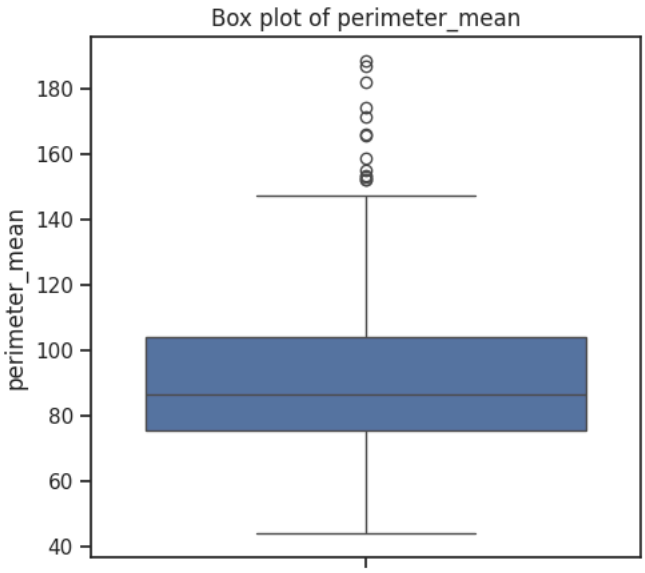

Histogram of radius_mean
Count
radius_mean

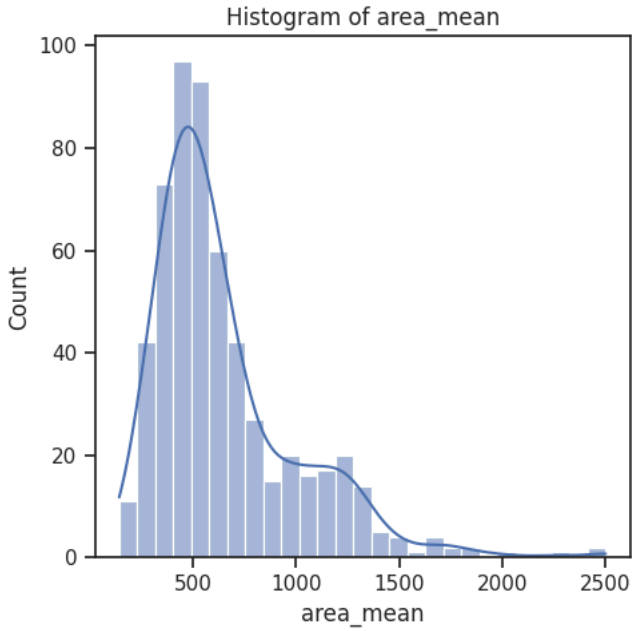

Box plot of radius_mean
radius_mean

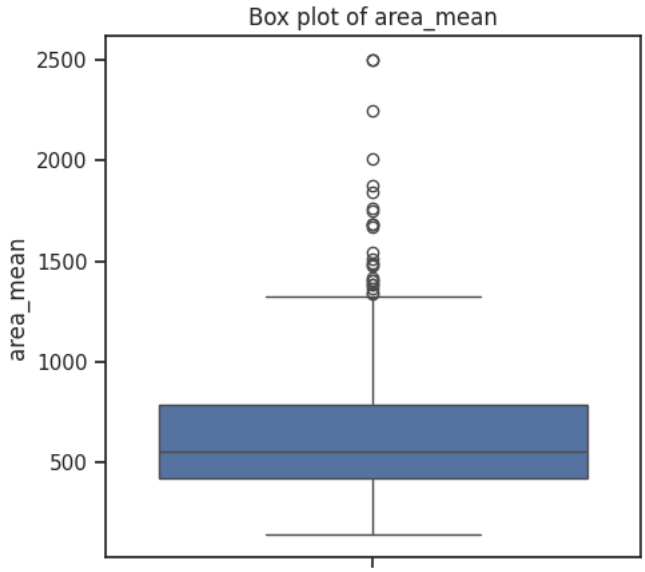






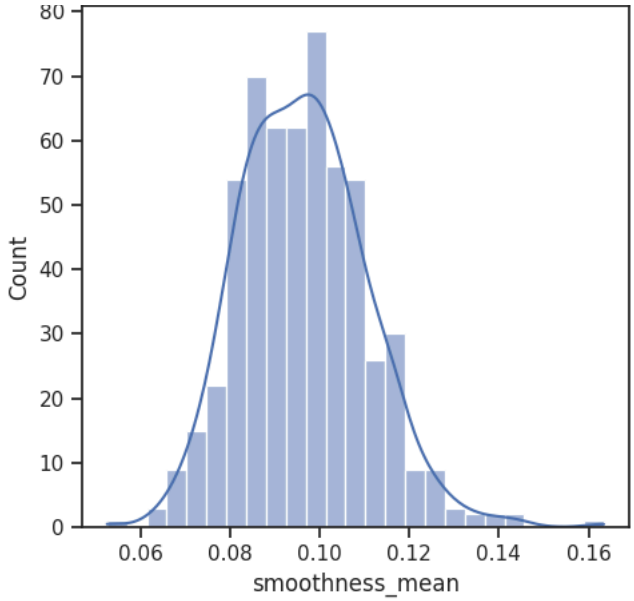


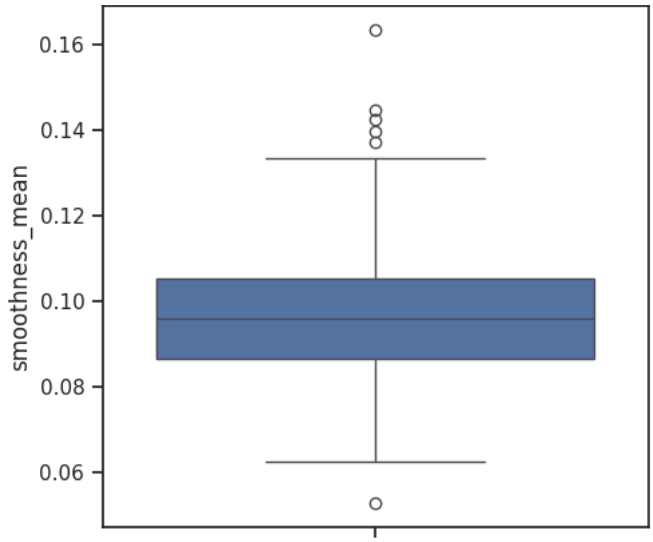




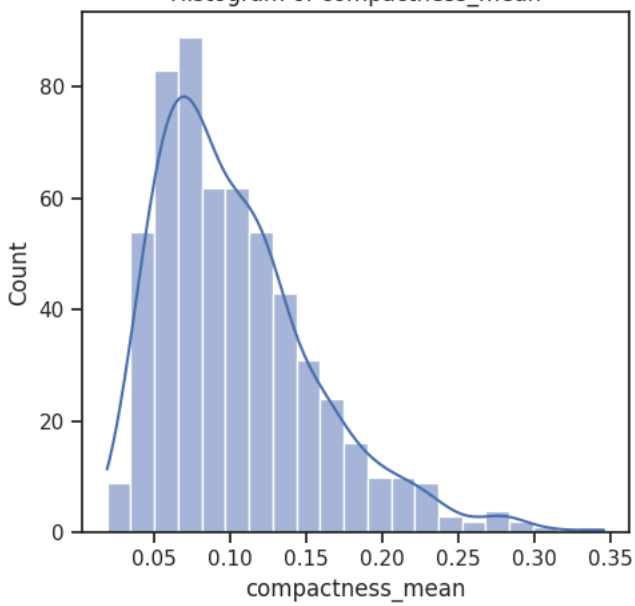
Histogram of smoothness_mean

Box plot of smoothness_mean

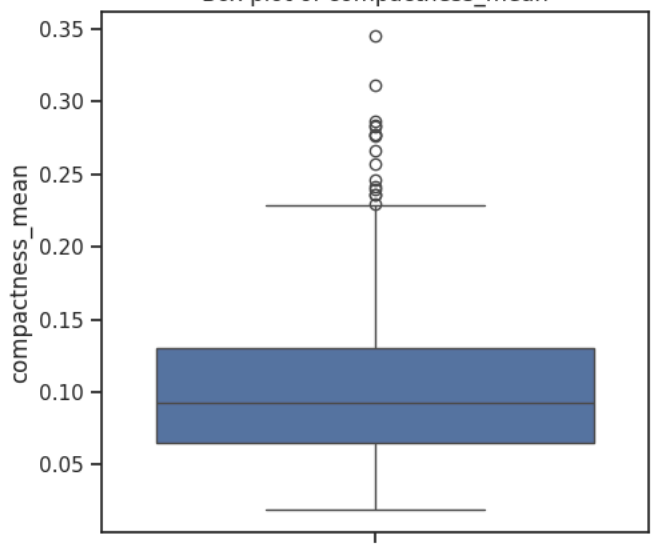




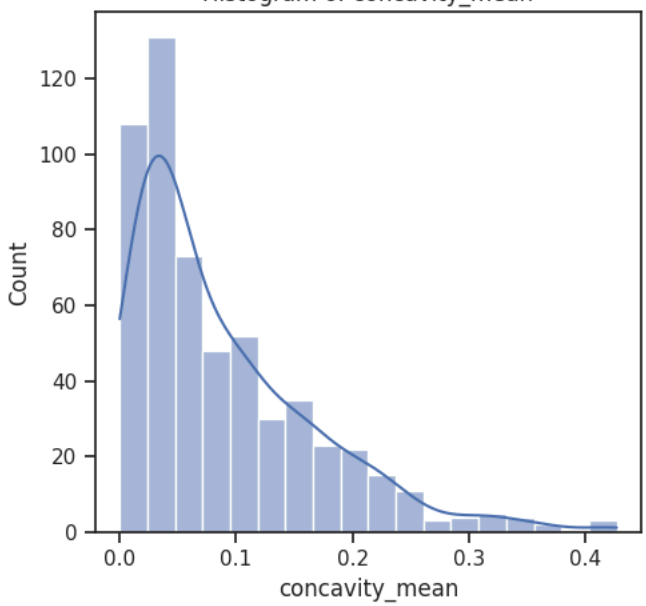
Histogram of compactness_mean



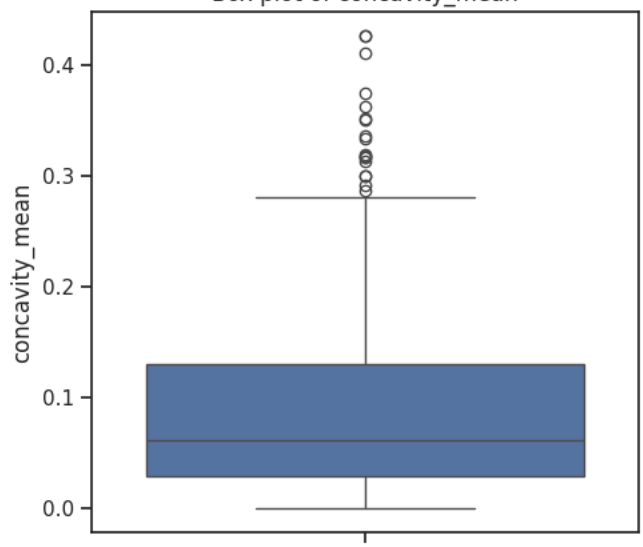
Box plot of compactness_mean



Histogram of concavity_mean

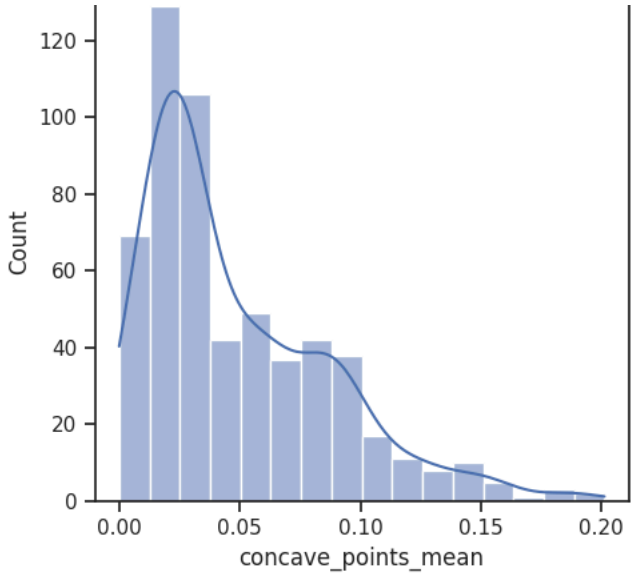


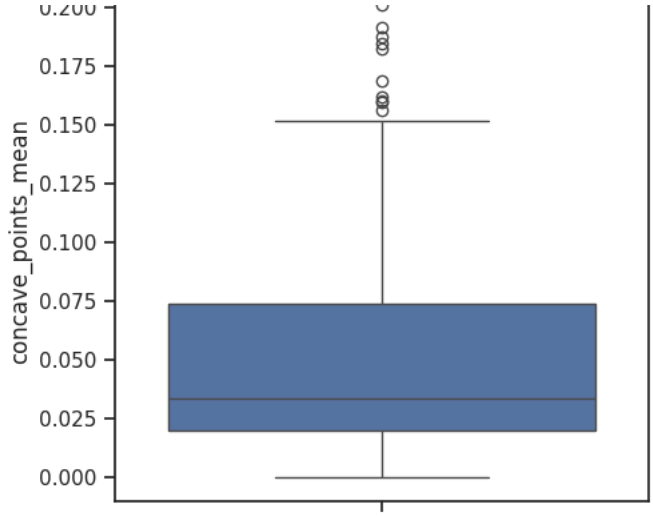
Box plot of concavity_mean

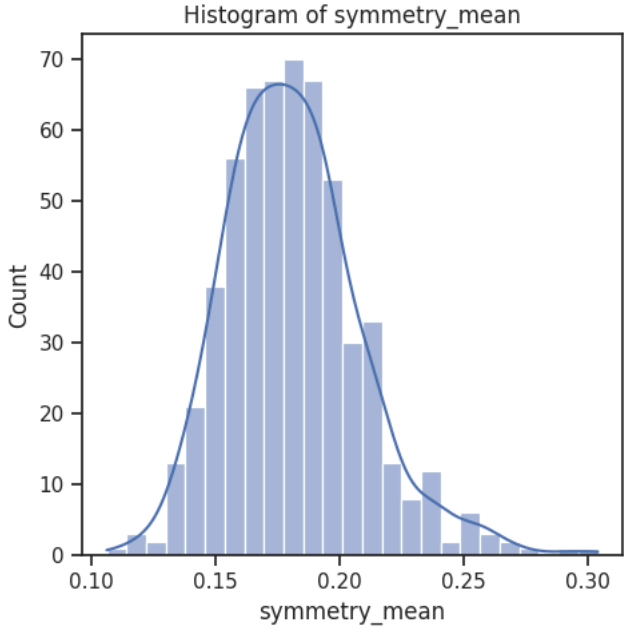


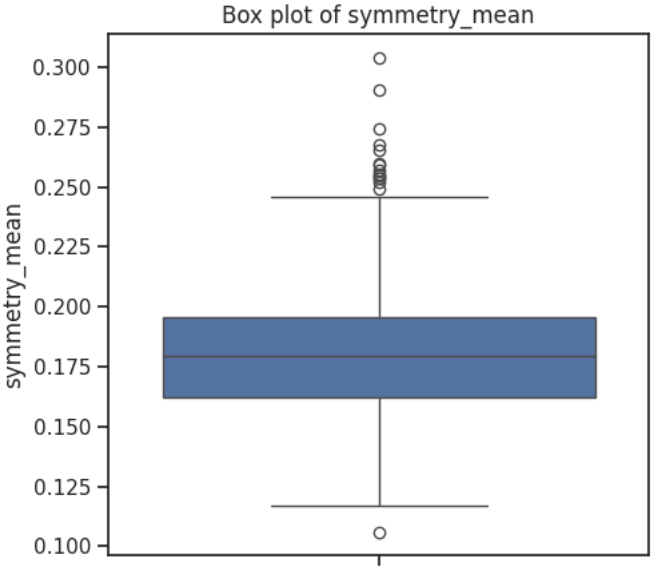
Histogram of concave_points_mean

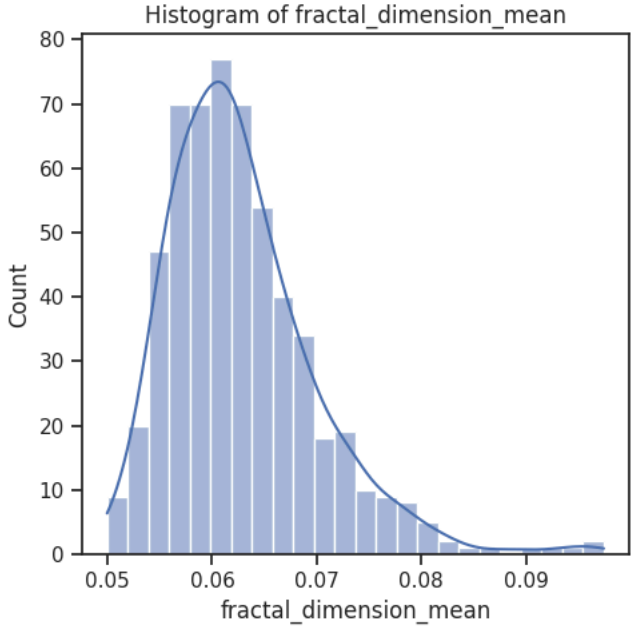
Box plot of concave_points_mean

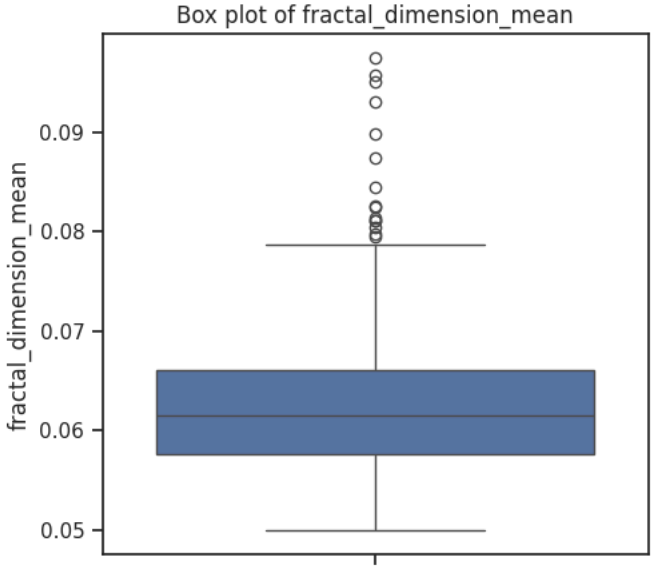

Count
concave_points_mean

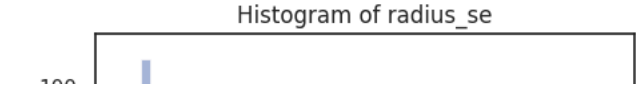

concave_points_mean

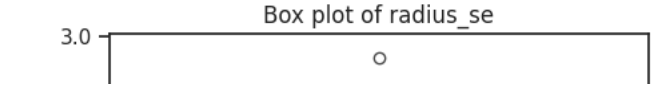

Histogram of symmetry_mean
Count
symmetry_mean

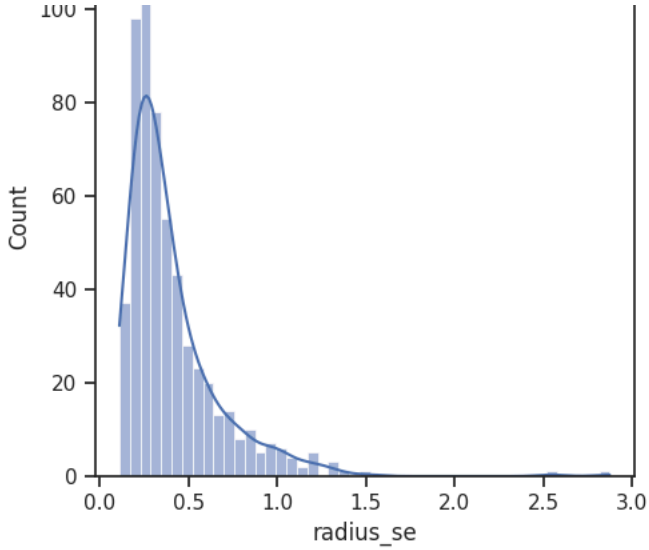

Box plot of symmetry_mean
symmetry_mean

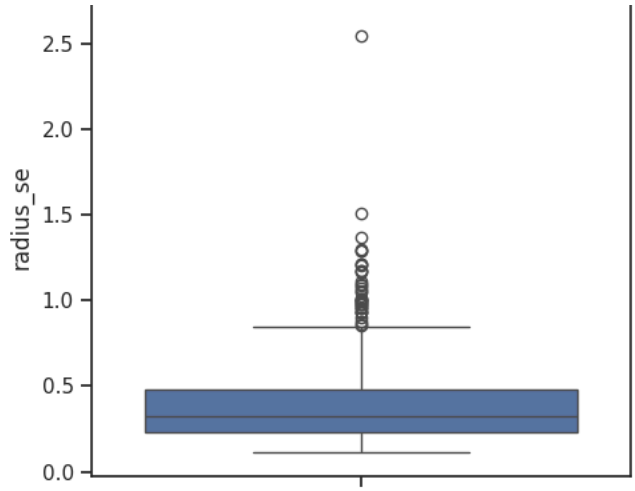

Histogram of fractal_dimension_mean
Count
fractal_dimension_mean

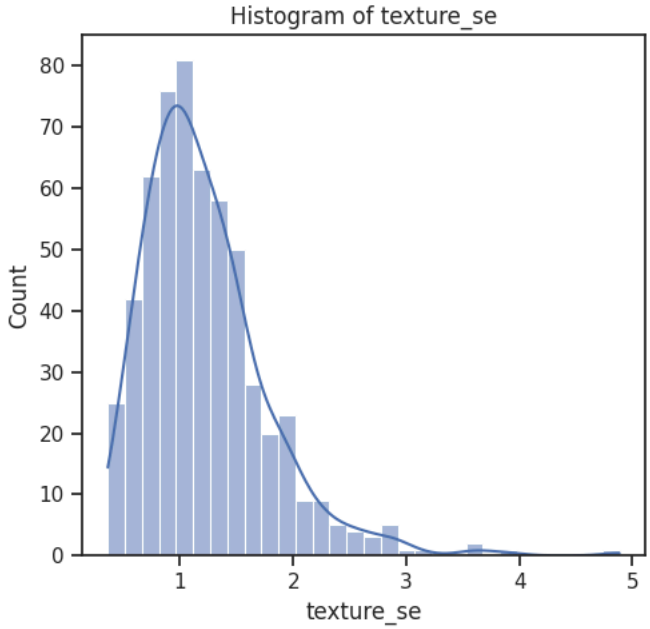

Box plot of fractal_dimension_mean
fractal_dimension_mean

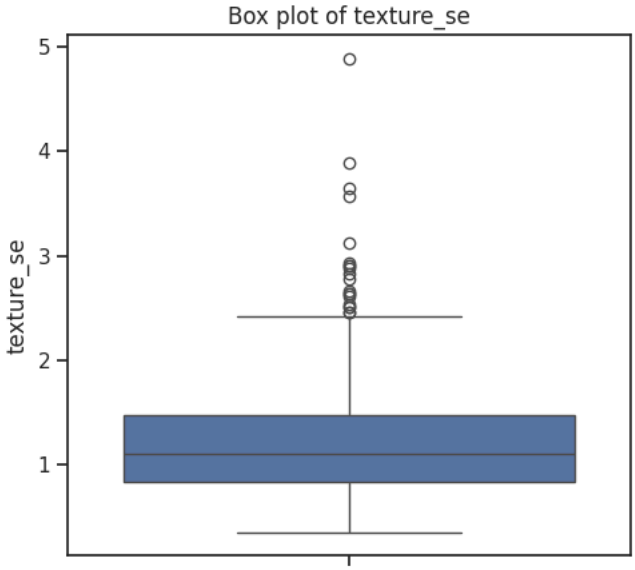

Histogram of radius_se

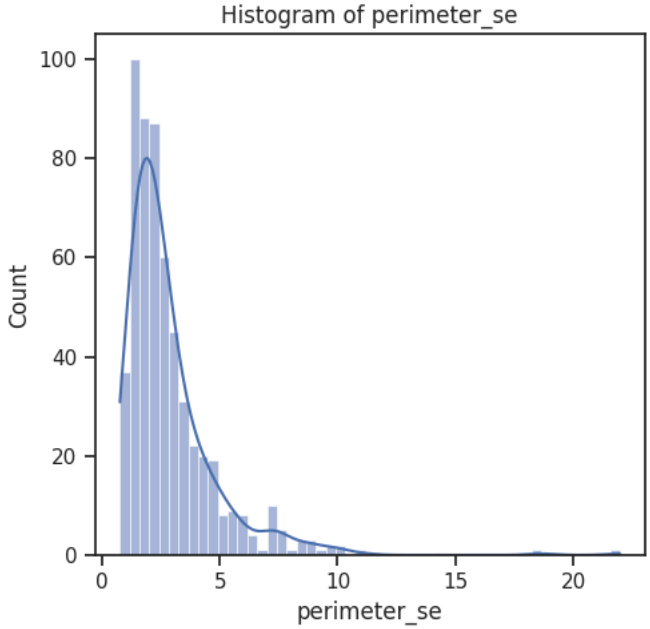

Box plot of radius_se

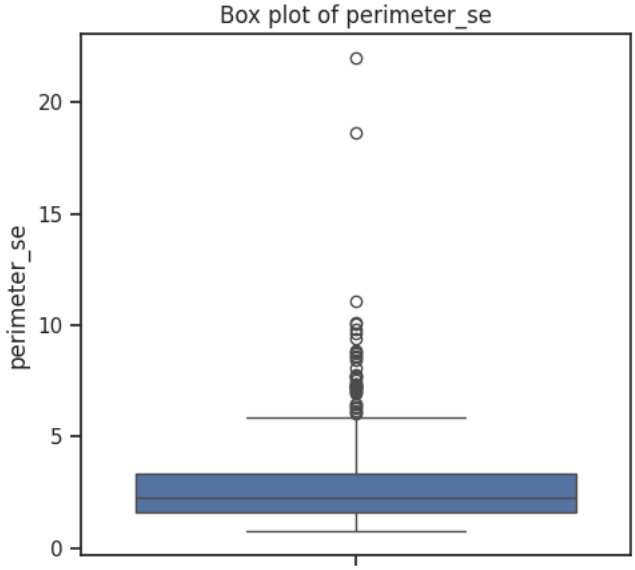

Count
radius_se

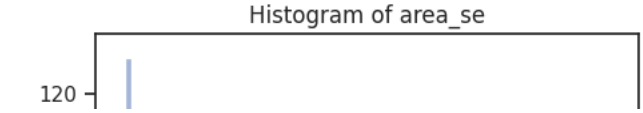

radius_se

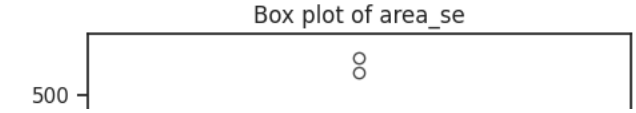

Histogram of texture_se
Count
texture_se

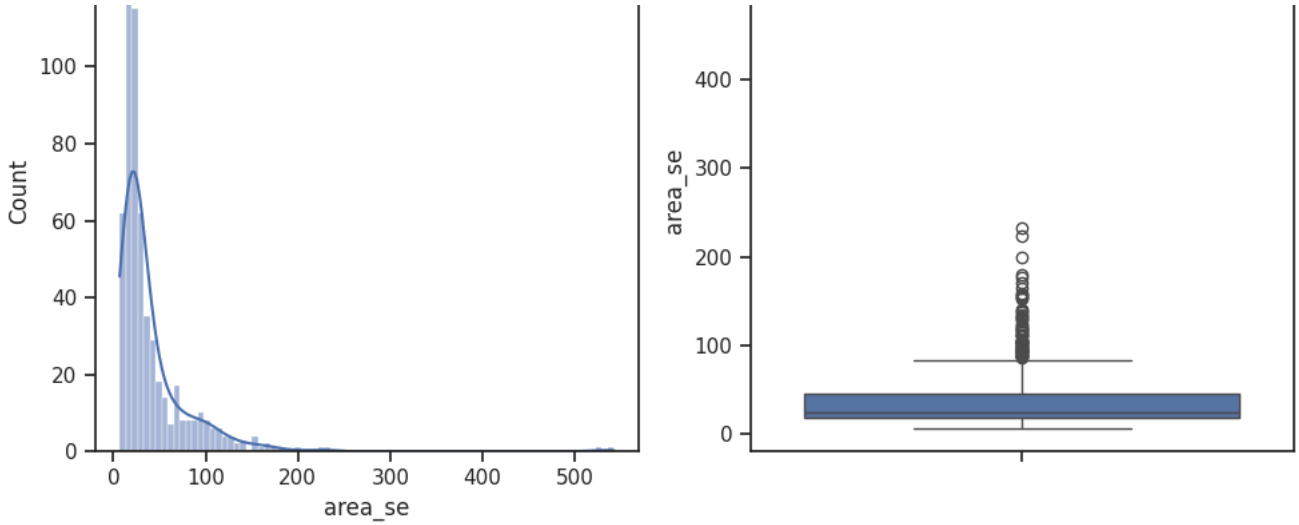

Box plot of texture_se
texture_se

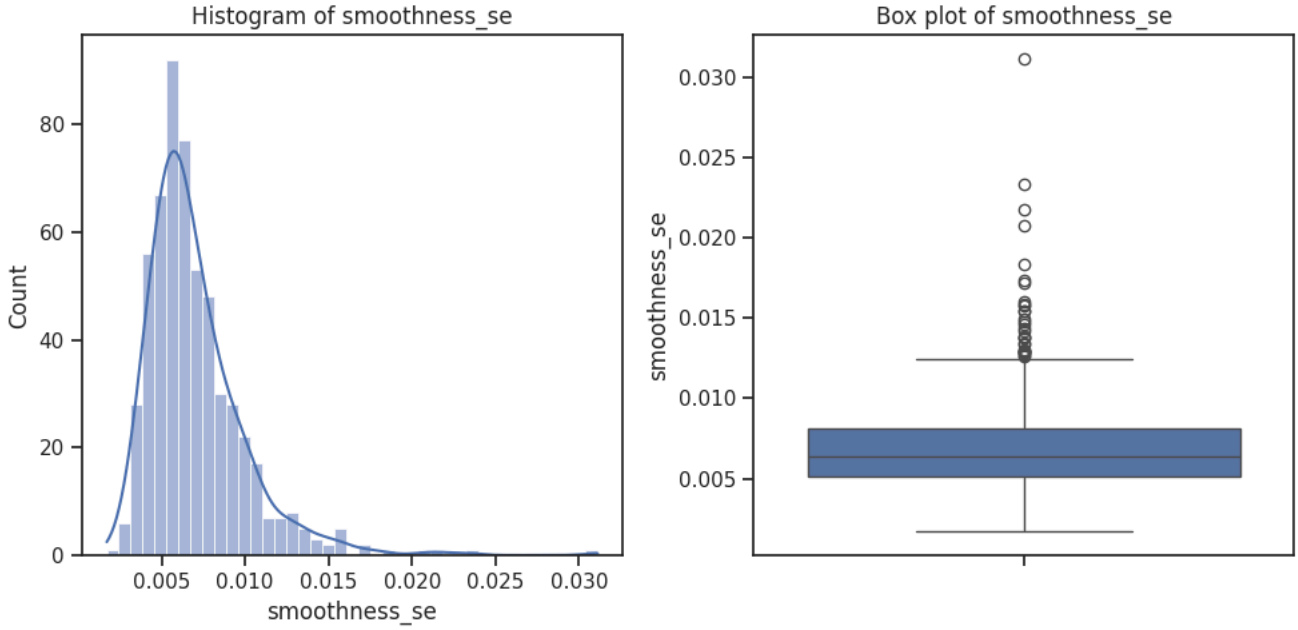

Histogram of perimeter_se
Count
perimeter_se

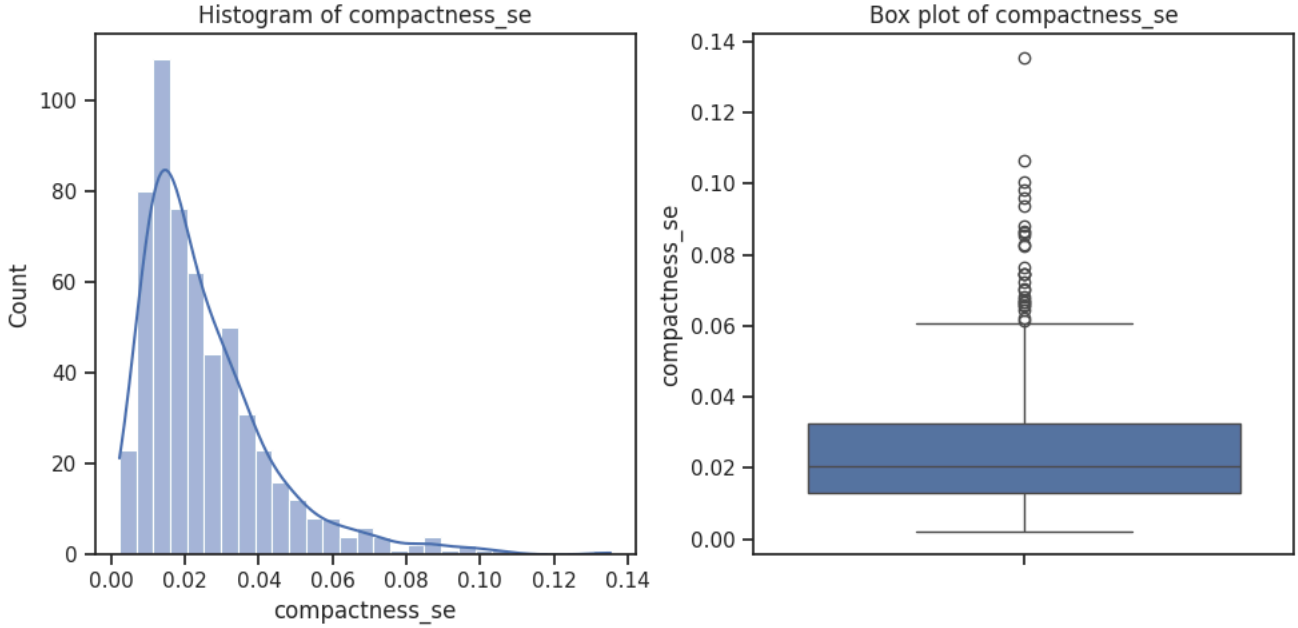

Box plot of perimeter_se
perimeter_se


Histogram of area_se

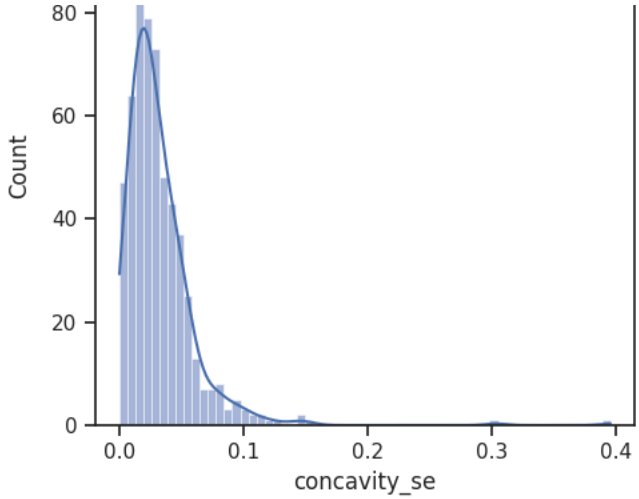

Box plot of area_se

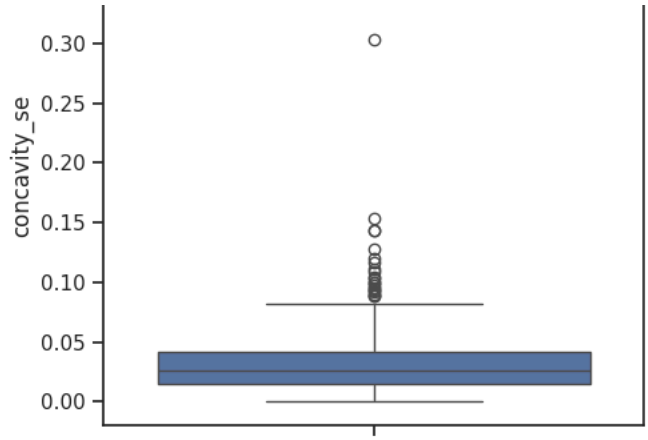

Count
area_se
area_se

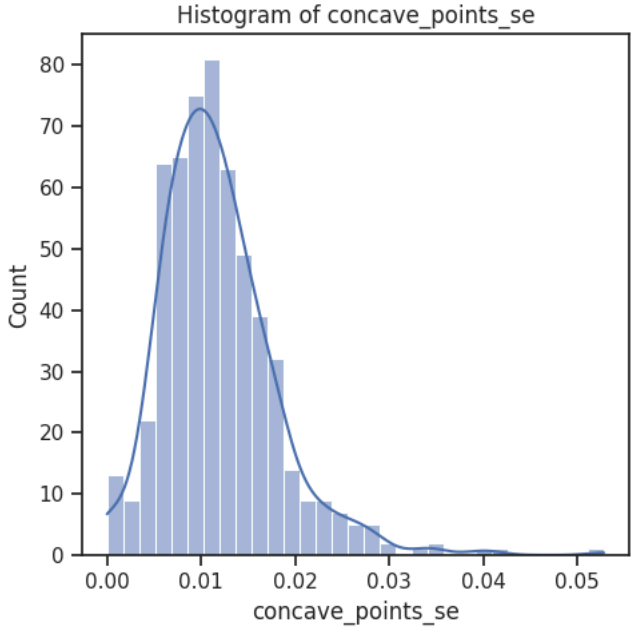

Histogram of smoothness_se
Box plot of smoothness_se
Count
smoothness_se
smoothness_se

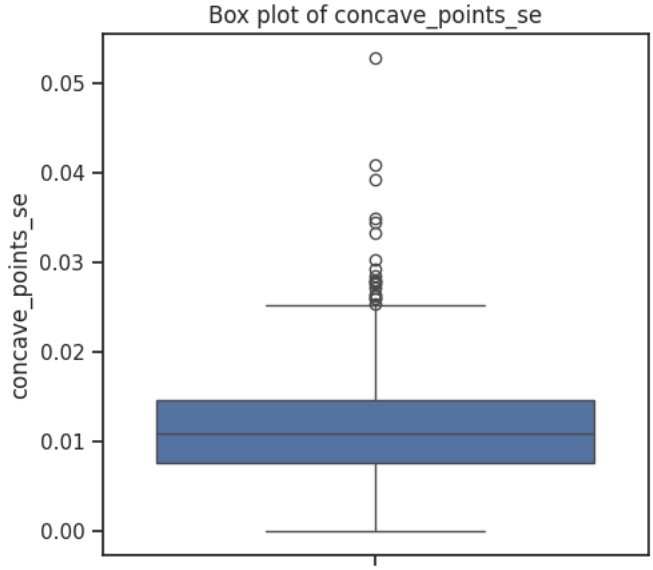

Histogram of compactness_se
Box plot of compactness_se
Count
compactness_se
compactness_se

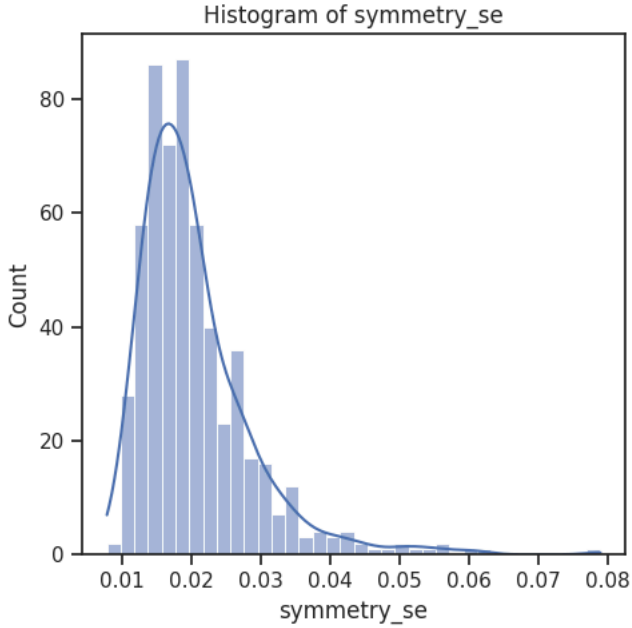
Histogram of concavity_se
Box plot of concavity_se

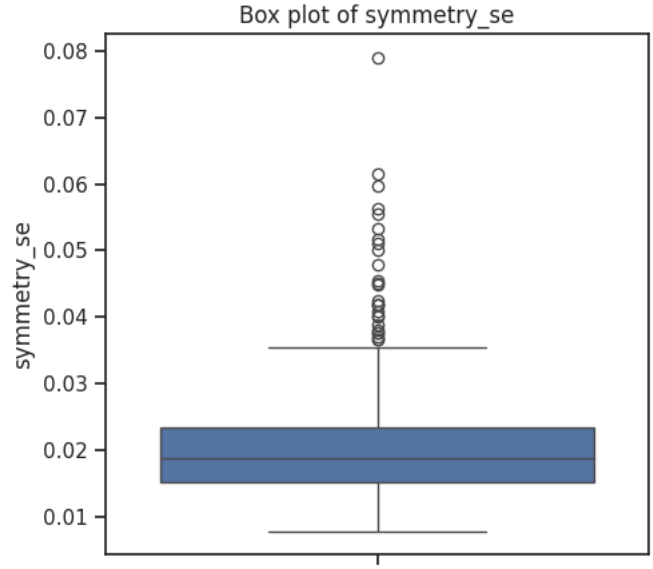

Count
concavity_se

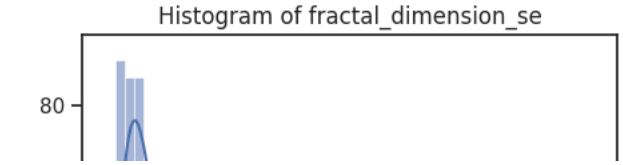

concavity_se

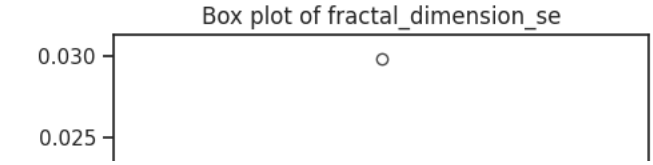

Histogram of concave_points_se
Count
concave_points_se

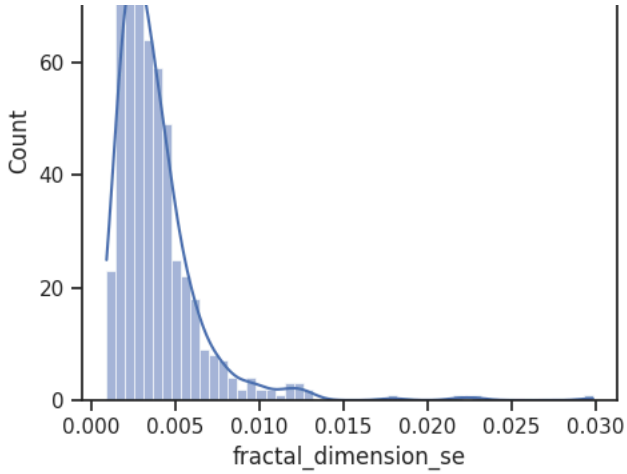

Box plot of concave_points_se
concave_points_se

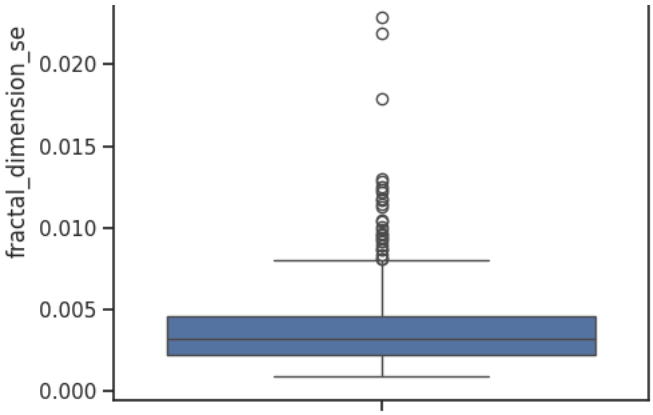

Histogram of symmetry_se
Count
symmetry_se

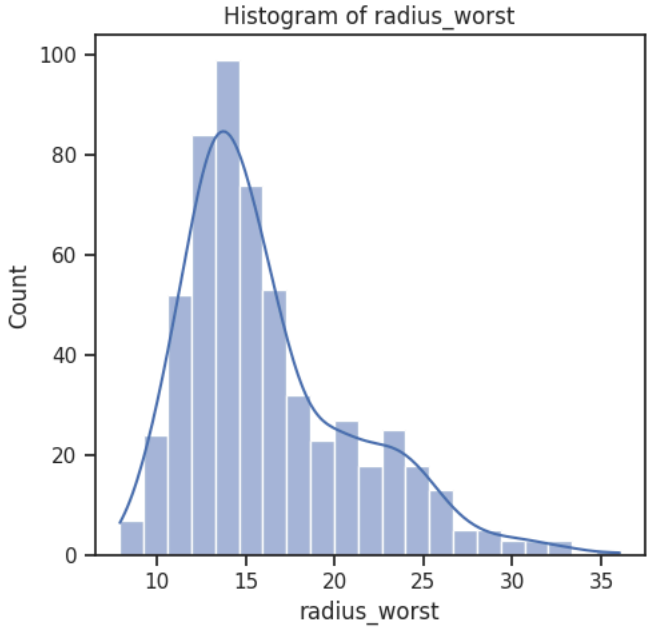

Box plot of symmetry_se
symmetry_se

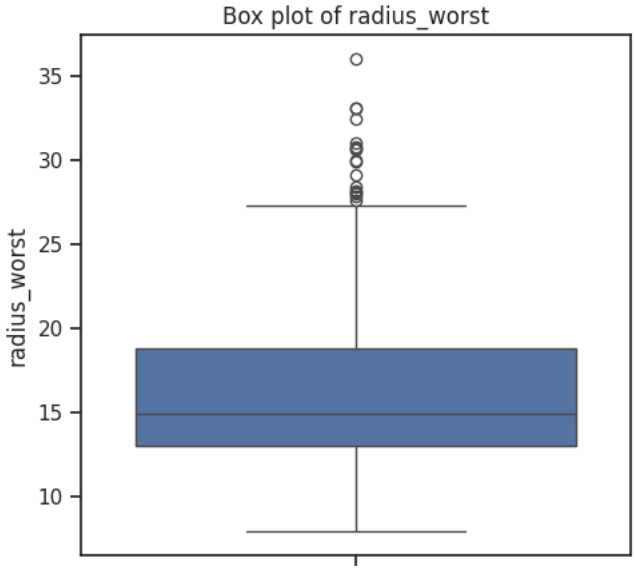

Histogram of fractal_dimension_se

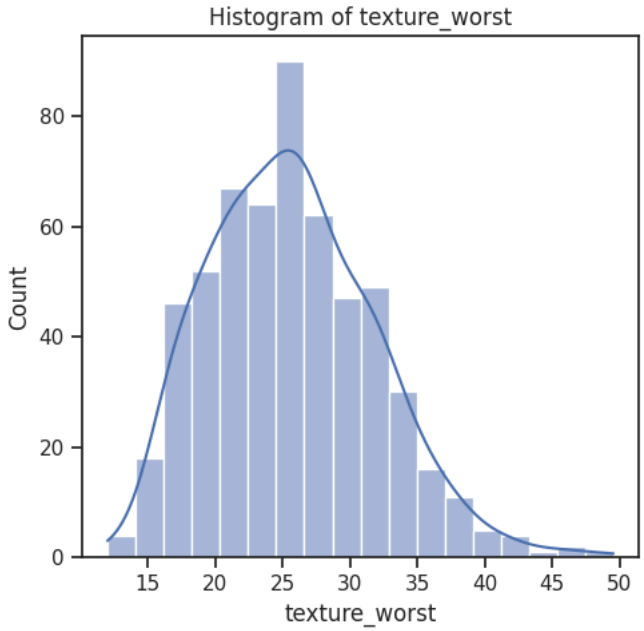

Box plot of fractal_dimension_se

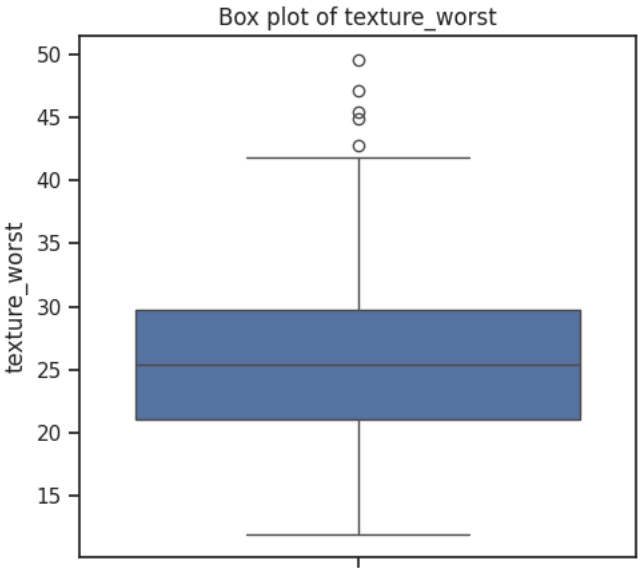

Count
fractal_dimension_se

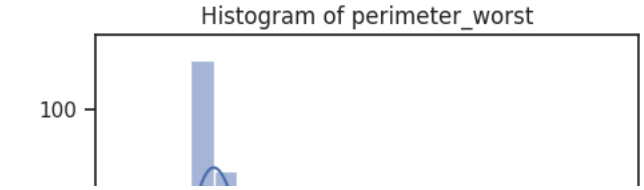

fractal_dimension_se

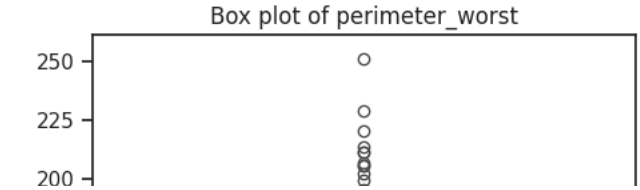

Histogram of radius_worst
Count
radius_worst

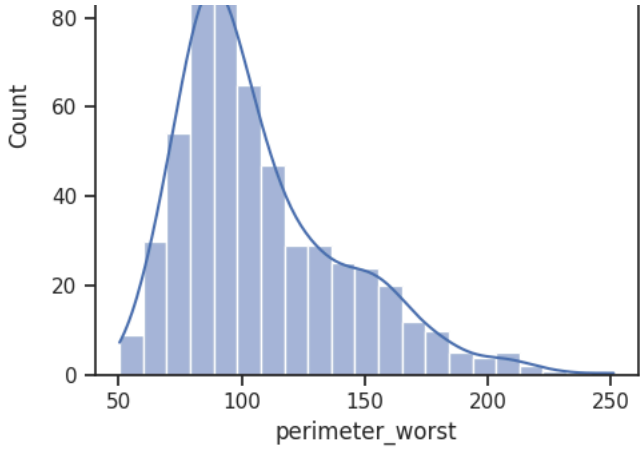

Box plot of radius_worst
radius_worst

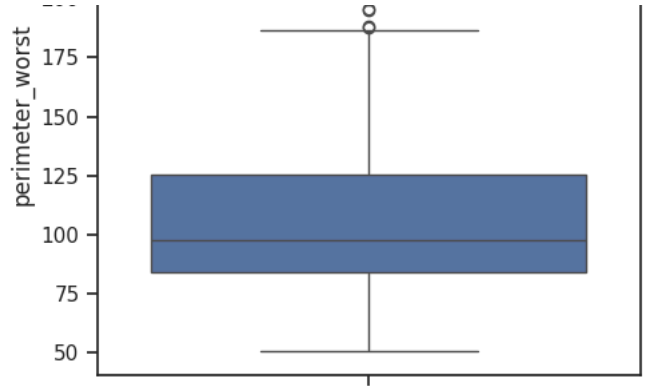

Histogram of texture_worst
Count
texture_worst

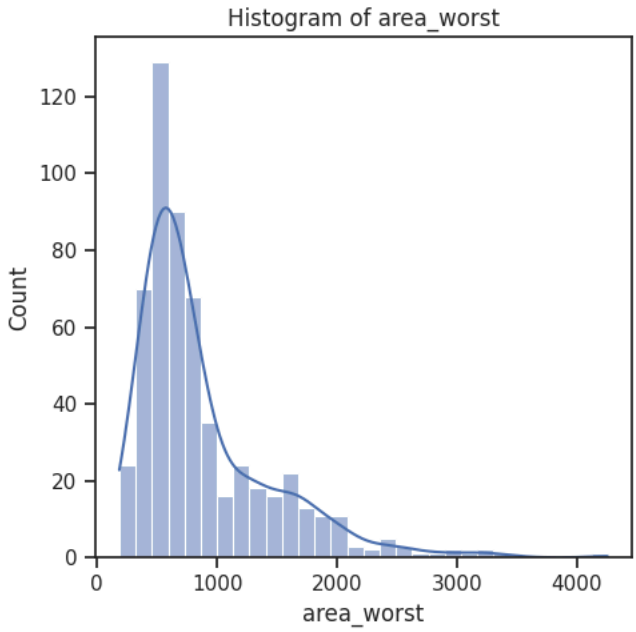

Box plot of texture_worst
texture_worst

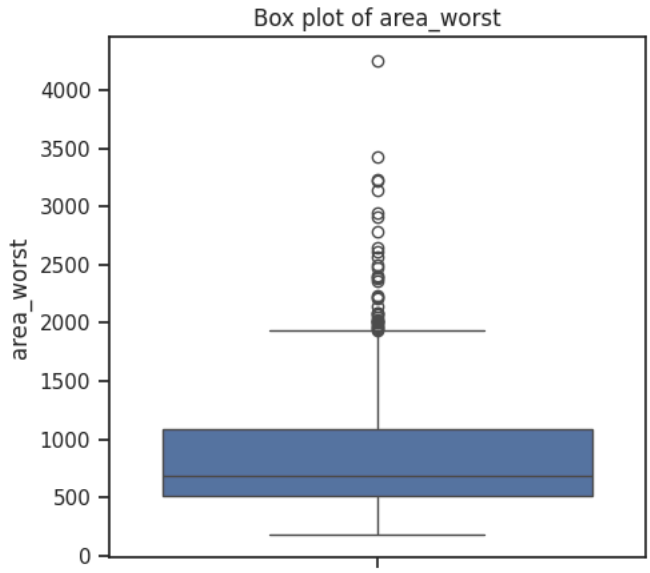

Histogram of perimeter_worst

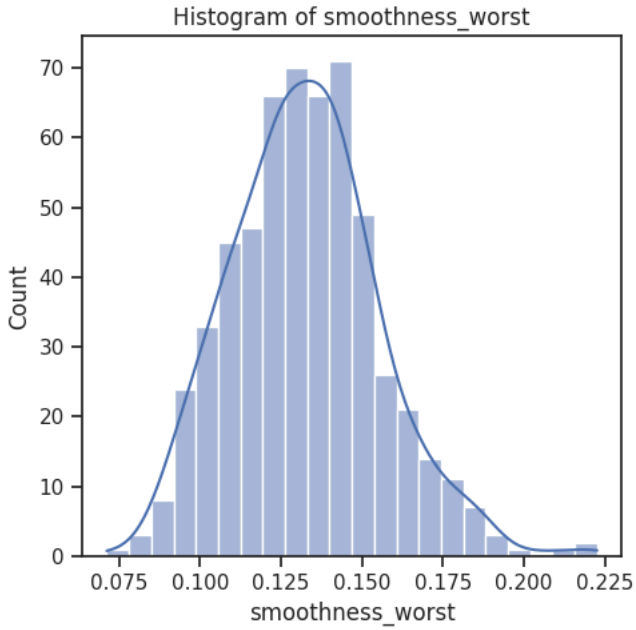

Box plot of perimeter_worst

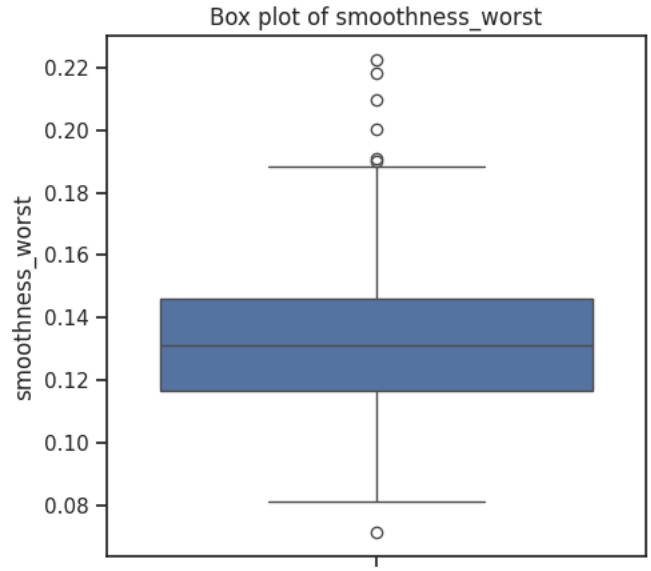

Count
perimeter_worst

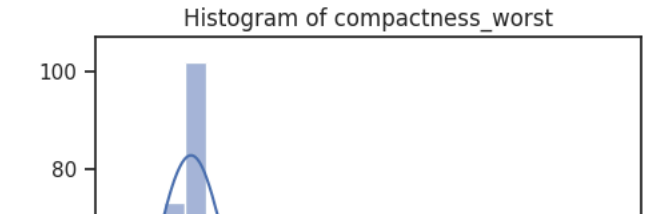

perimeter_worst

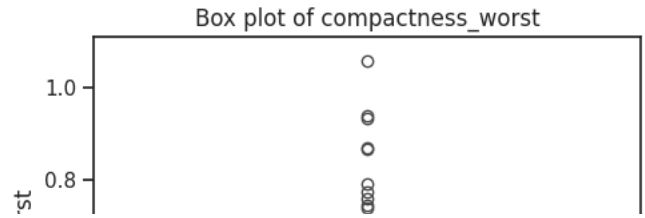

Histogram of area_worst
Count
area_worst

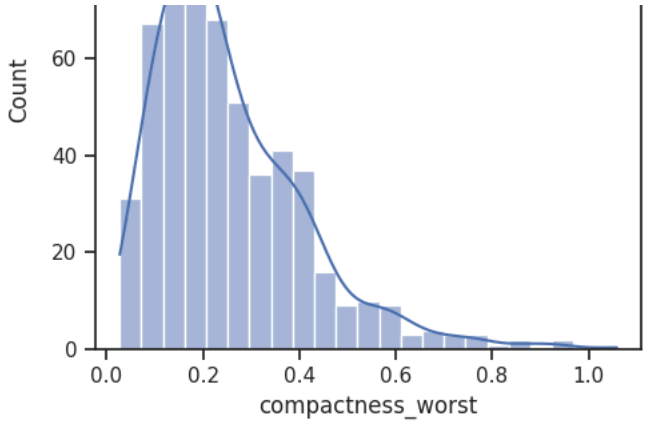

Box plot of area_worst
area_worst

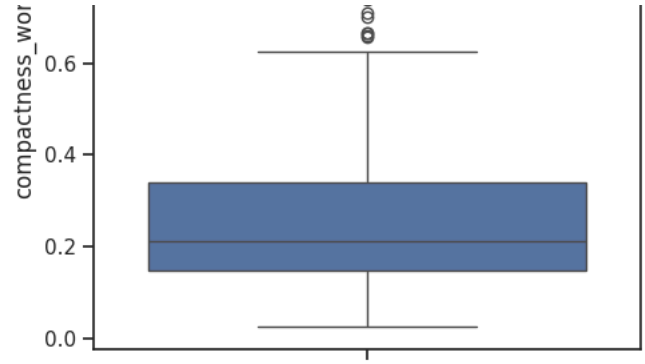

Histogram of smoothness_worst
Count
smoothness_worst

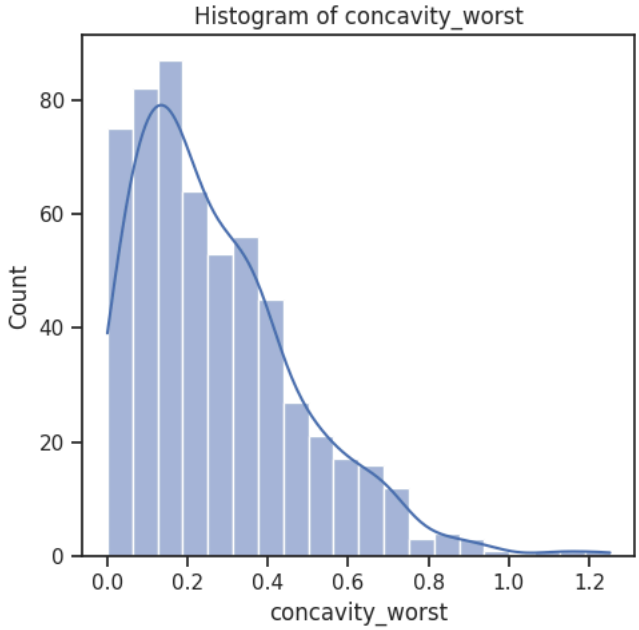

Box plot of smoothness_worst
smoothness_worst

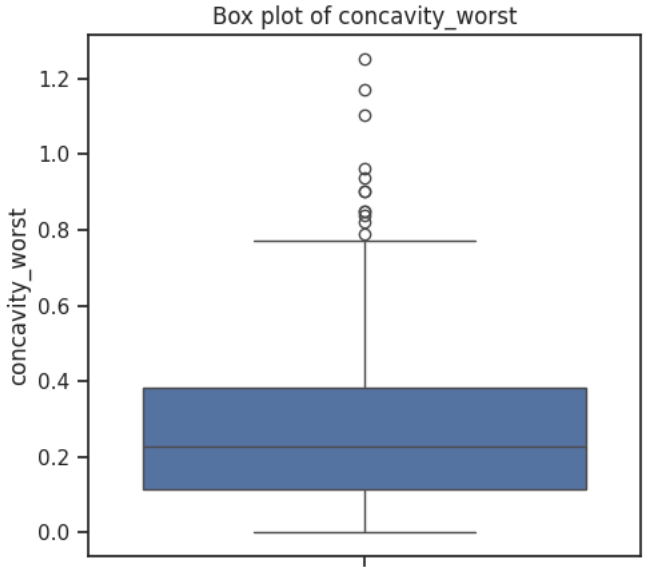

Histogram of compactness_worst

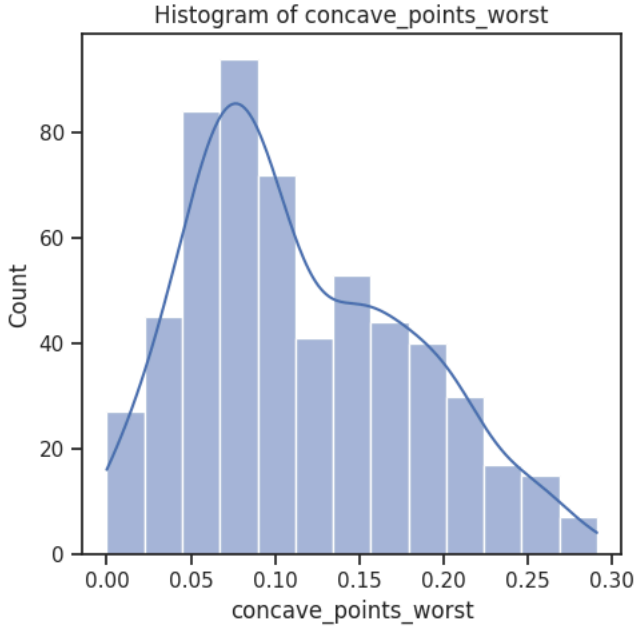

Box plot of compactness_worst

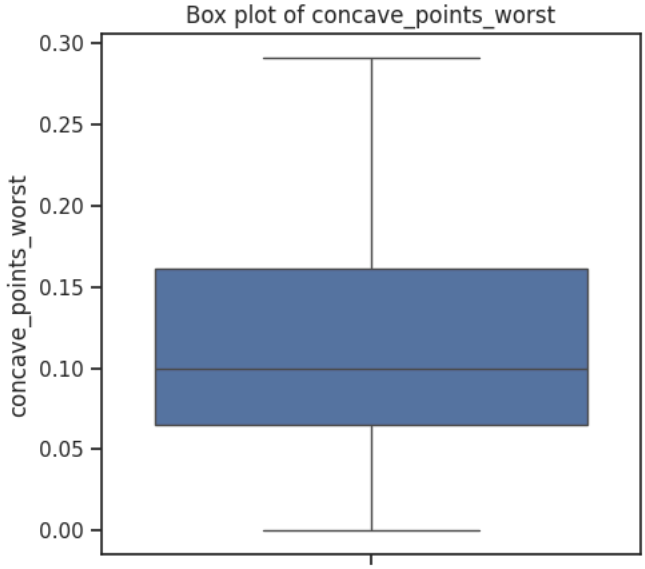

Count
compactness_worst

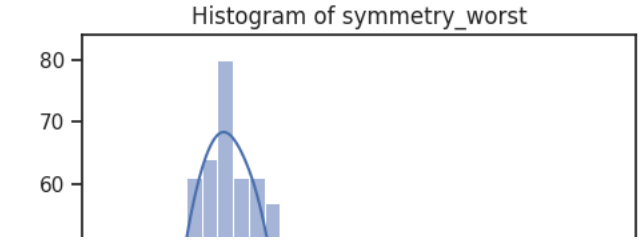

compactness_wor

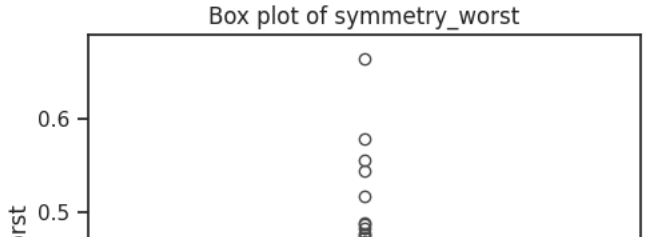

Histogram of concavity_worst
Count
concavity_worst

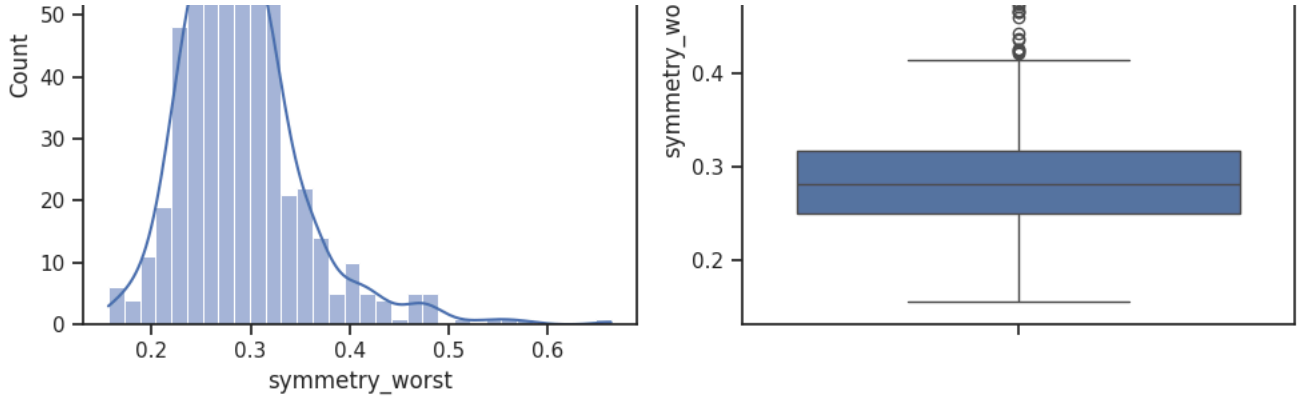

Box plot of concavity_worst
concavity_worst

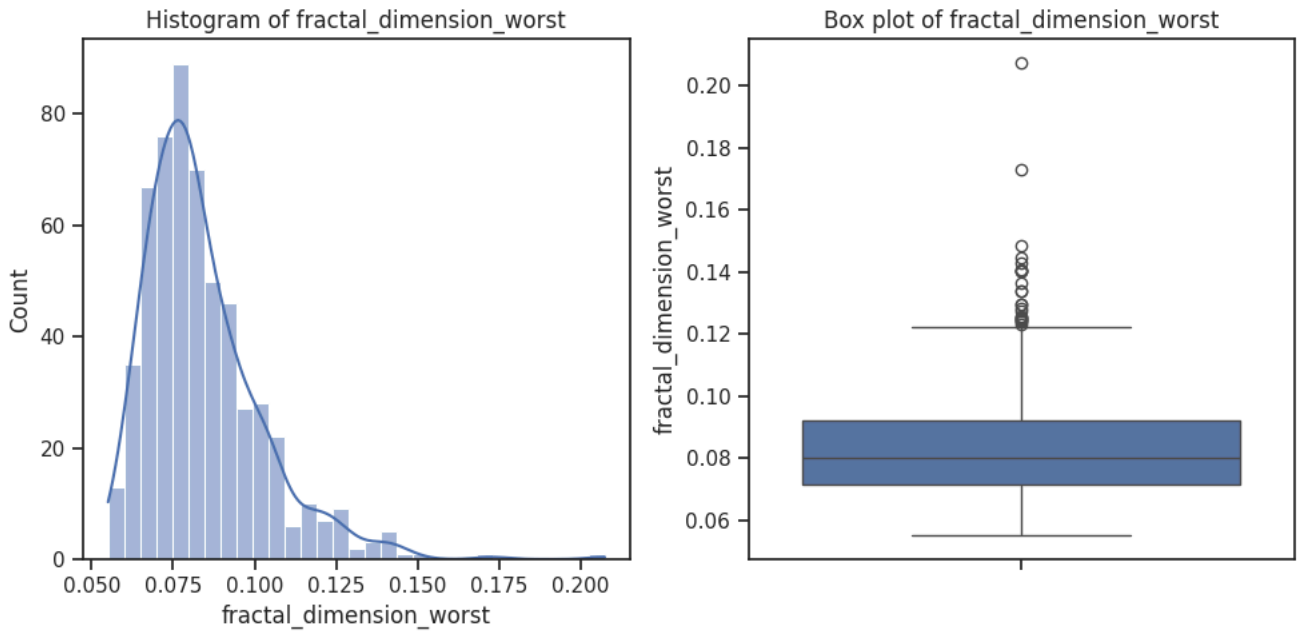

Histogram of concave_points_worst
Count
concave_points_worst


Box plot of concave_points_worst
concave_points_worst


Histogram of symmetry_worst


Box plot of symmetry_worst





From the above graphical representations, we can infer that almost all the numerical values are normally distributed except for few and the all the column has outliers.

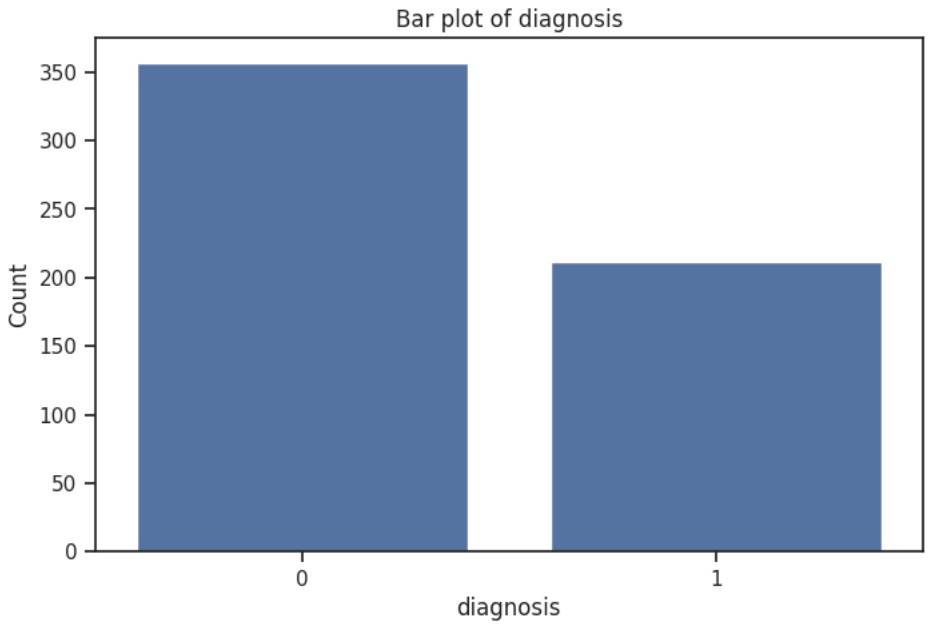
In the below code, we are seperating the categorical variable. and storing it in a variable.

```
# Select categorical columns for analysis (assuming 'diagnosis' column is categorical)
categorical_cols = ['diagnosis']
```

The below code effectively generates both frequency tables and bar plots for each categorical column, allowing for a visual and quantitative understanding of the distribution of categories within the dataset.

```
# Display frequency tables showing counts and percentages
for col in categorical_cols:
    frequency_table = df[col].value_counts(normalize=True) * 100
    print(f"Frequency table for {col}:")
    print(frequency_table)
    print("\n")

    # Visualize using bar plots
    plt.figure(figsize=(8, 5))
    sns.countplot(data=df, x=col)
    plt.title(f'Bar plot of {col}')
    plt.xlabel(col)
    plt.ylabel('Count')
    plt.show()
```

```
Frequency table for diagnosis:
diagnosis
0    62.741652
1    37.258348
Name: proportion, dtype: float64
```



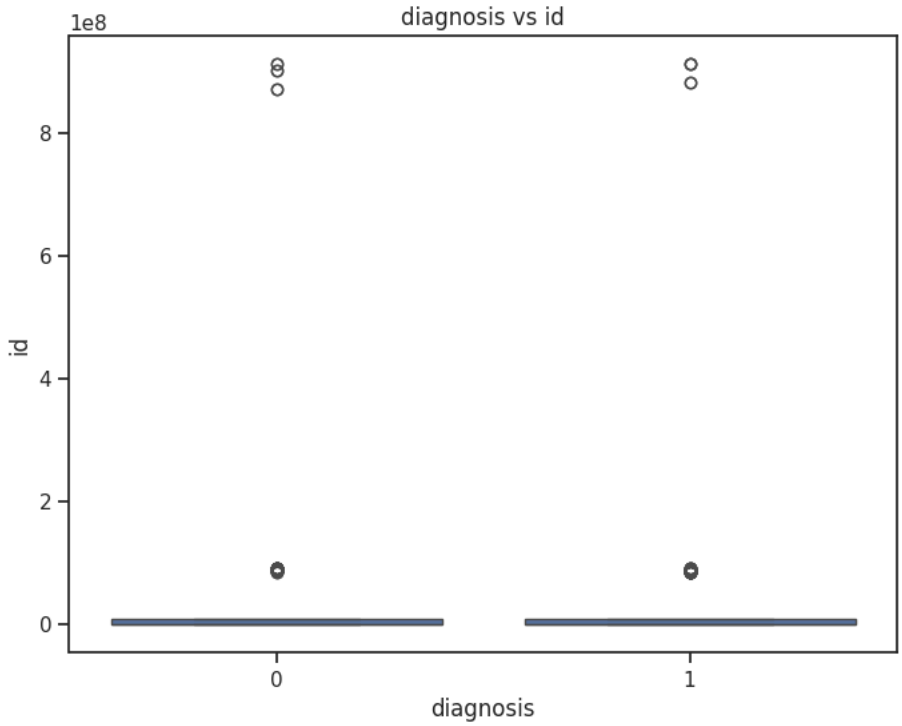
- The provided frequency table is for the 'diagnosis' column, which contains categorical values indicating the diagnosis of tumors. Here's an explanation of the result:
  - diagnosis: This column represents the diagnosis of tumors, with '0' indicating benign tumors and '1' indicating malignant tumors.
- The frequency table shows the proportion of each diagnosis category within the column:
  - Category '0' (benign tumors) accounts for approximately 62.74% of the diagnoses.
  - Category '1' (malignant tumors) accounts for approximately 37.26% of the diagnoses.
- This indicates that in the dataset:
  - Around 62.74% of tumors are diagnosed as benign.
  - Around 37.26% of tumors are diagnosed as malignant.
- These percentages provide insights into the distribution of diagnoses within the dataset, indicating the relative prevalence of benign and malignant tumors.
- From the graph we can infer that the count of malignant is high than Benign which means the no of people in serious condition is less compared to the people in initial stage of cancer.

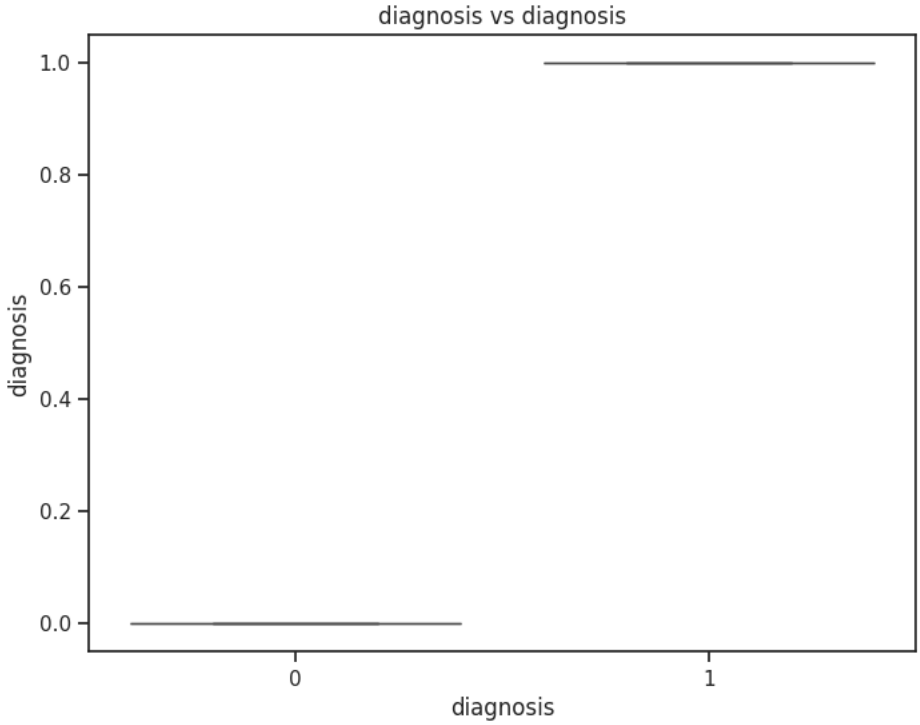
In the below code, we seperating the numerical variables and categorical variables and storing it in a variable.

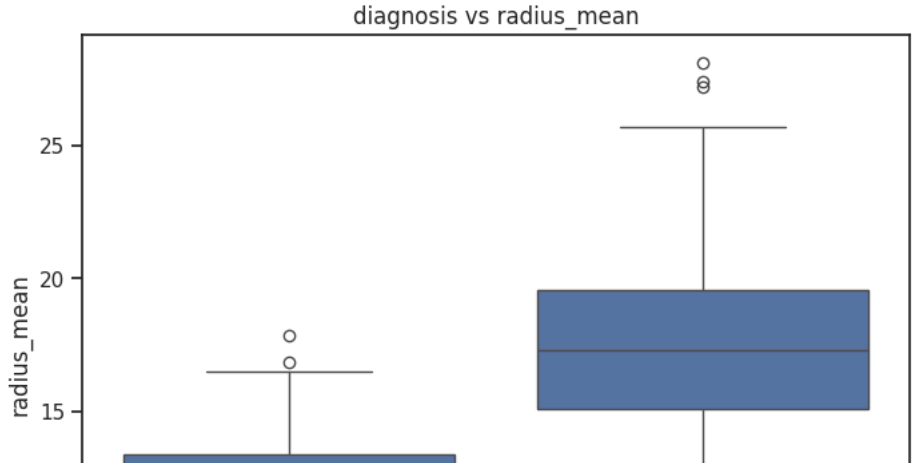
```
# Bivariate analysis between numerical and categorical variables using box plots or violin plots
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
categorical_cols = ['diagnosis']
```

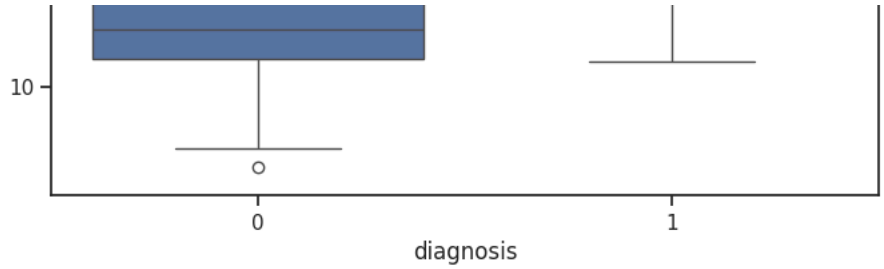
The below code allows for visual examination of how the distribution of numerical variables varies across different categories of categorical variables, providing insights into potential relationships or differences within the dataset.

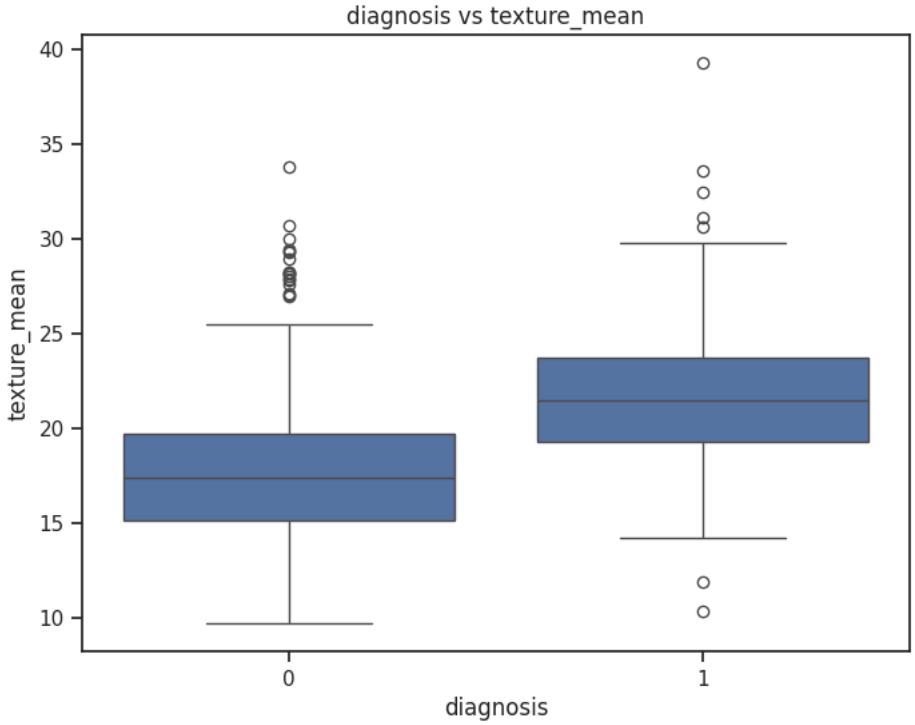
```
for num_col in numerical_cols:
    for cat_col in categorical_cols:
        plt.figure(figsize=(8, 6))
        sns.boxplot(data=df, x=cat_col, y=num_col)
        plt.title(f'{cat_col} vs {num_col}')
        plt.xlabel(cat_col)
        plt.ylabel(num_col)
        plt.show()
```

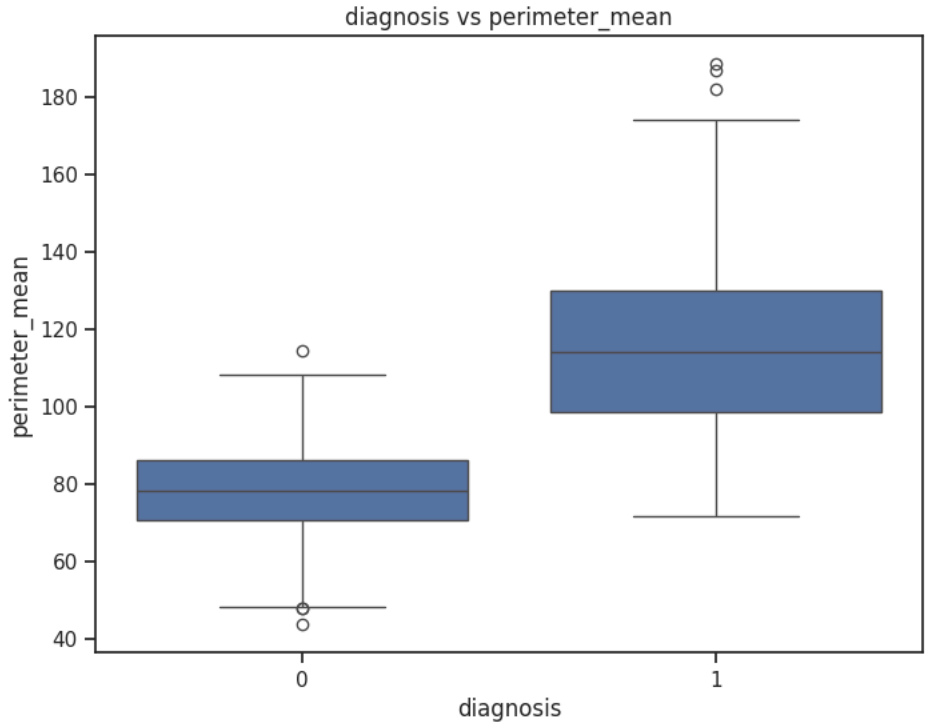

diagnosis vs id
1e8
id
diagnosis
0
1
0
2
4
6
8

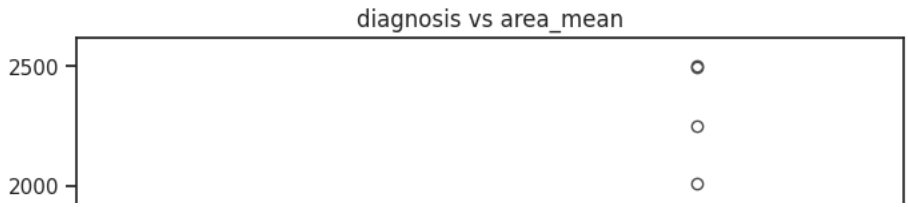

diagnosis vs diagnosis
diagnosis
diagnosis
0
1
0.0
0.2
0.4
0.6
0.8
1.0

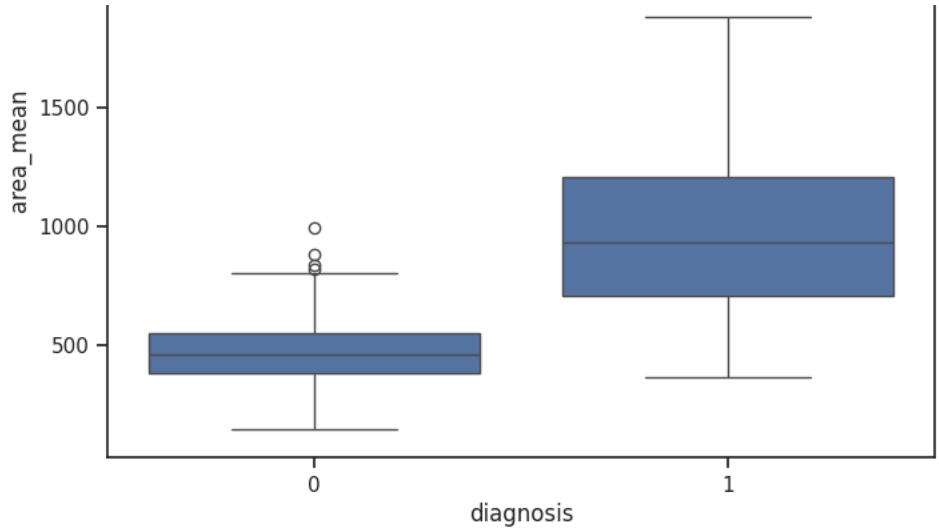

diagnosis vs radius_mean
radius_mean
15
20
25

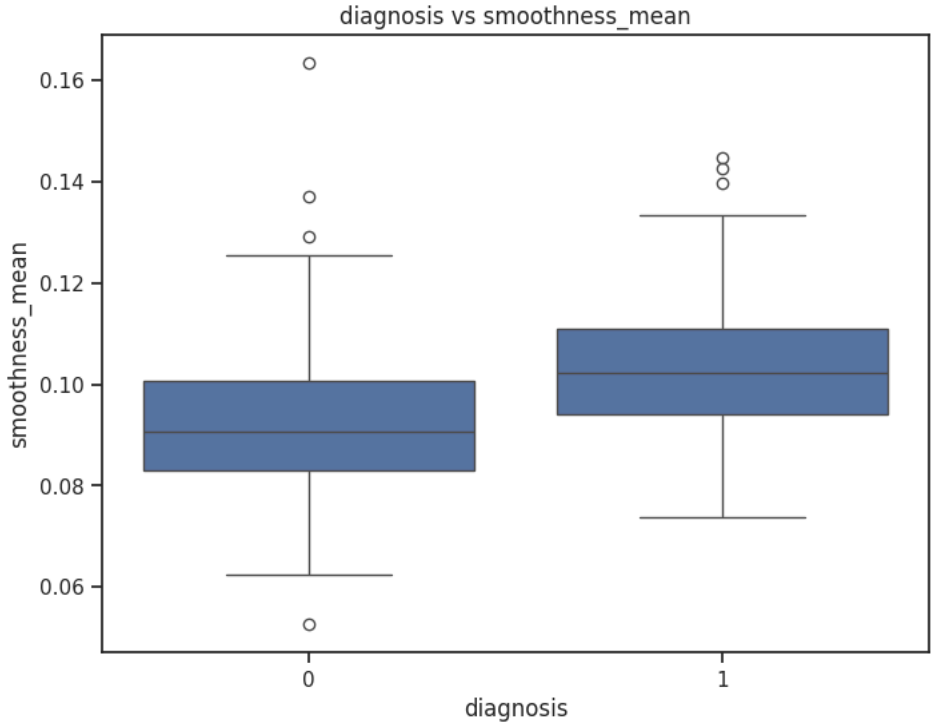

10
0
1
diagnosis

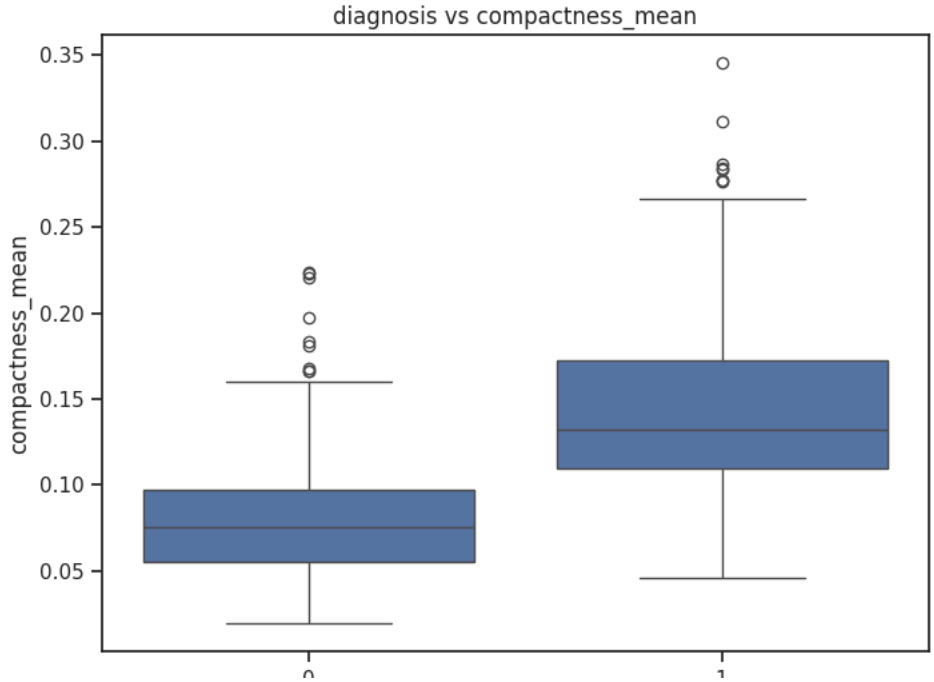

diagnosis vs texture_mean
texture_mean
40
35
30
25
20
15
10
0
1
diagnosis

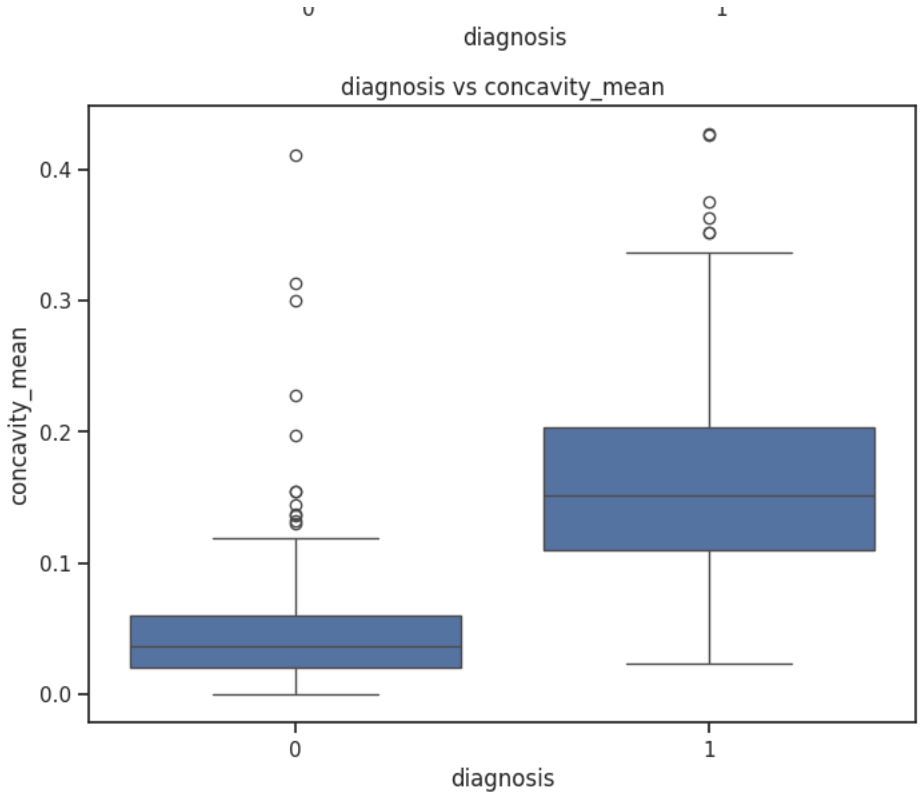

diagnosis vs perimeter_mean
perimeter_mean
180
160
140
120
100
80
60
40
0
1
diagnosis

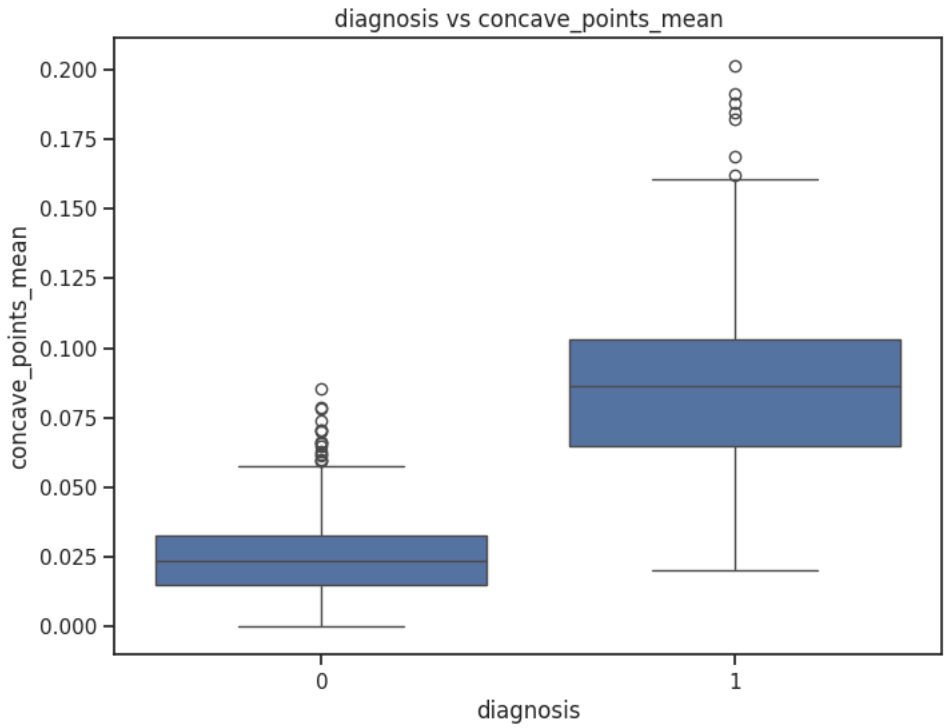

diagnosis vs area_mean
2500
2000

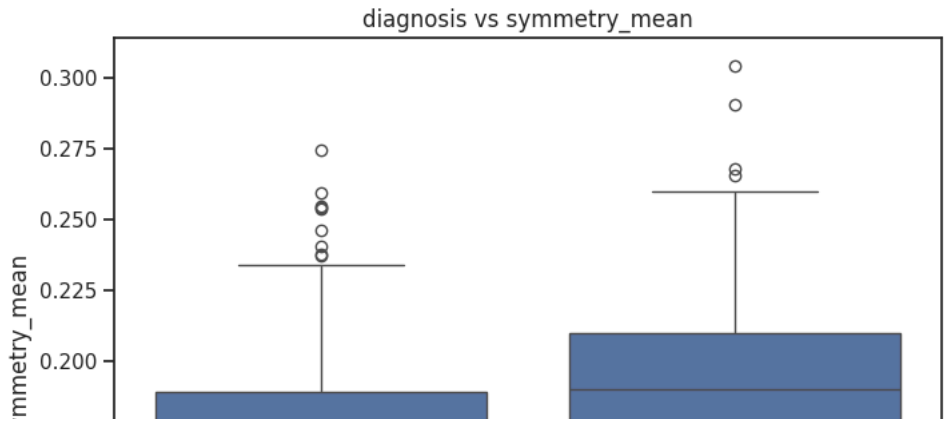

area_mean
1500
1000
500
0
1
diagnosis

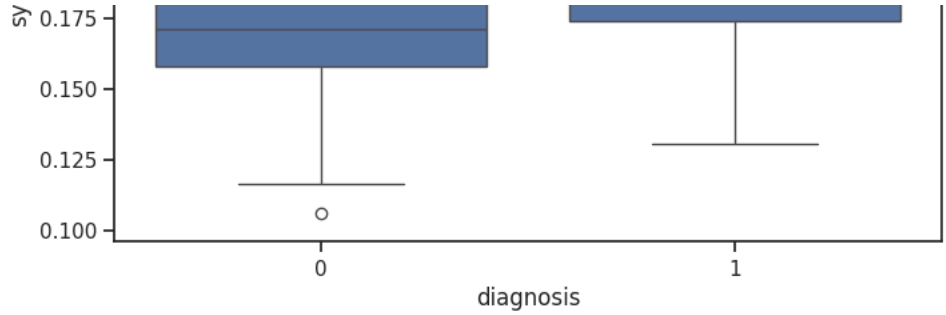

diagnosis vs smoothness_mean
smoothness_mean
0.16
0.14
0.12
0.10
0.08
0.06
0
1
diagnosis

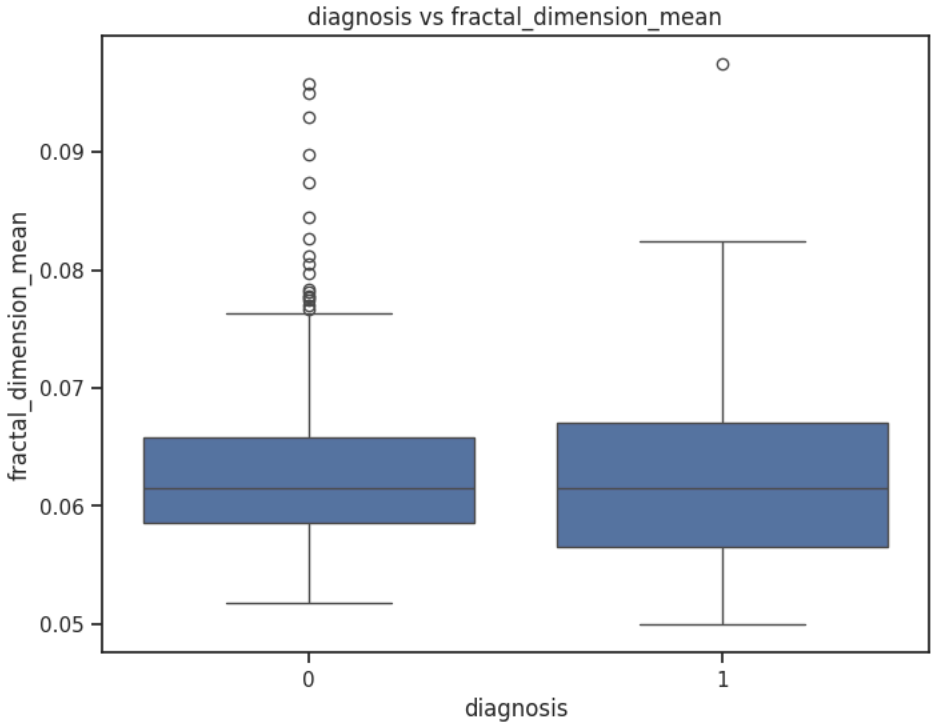

diagnosis vs compactness_mean
compactness_mean
0.35
0.30
0.25
0.20
0.15
0.10
0.05

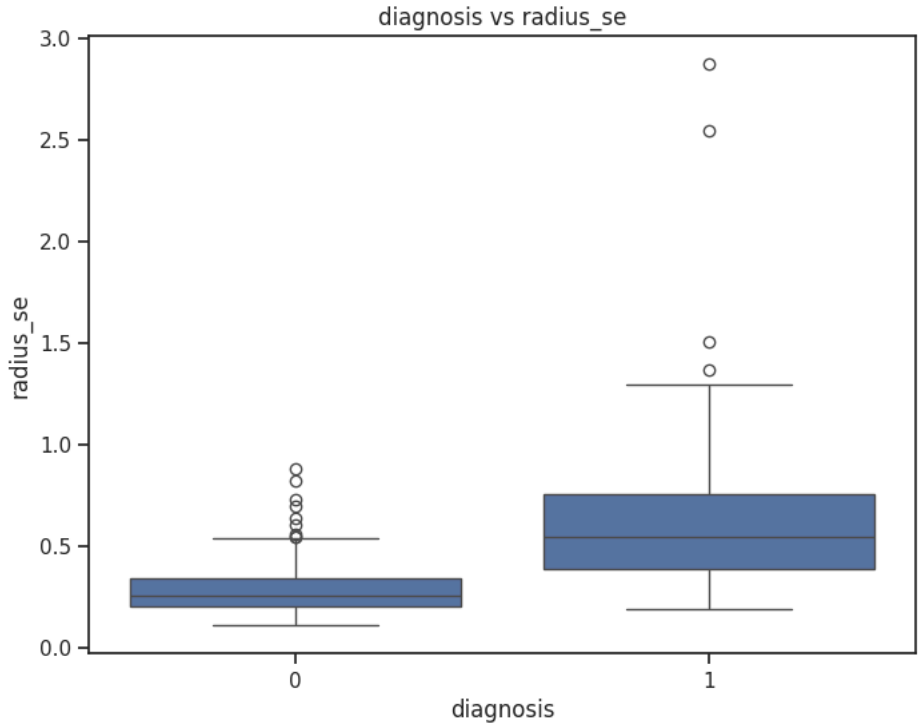

diagnosis
diagnosis vs concavity_mean
concavity_mean
0.0
0.1
0.2
0.3
0.4
0
1
diagnosis

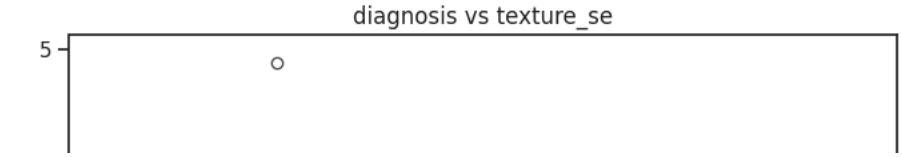

diagnosis vs concave_points_mean
concave_points_mean
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
0
1
diagnosis

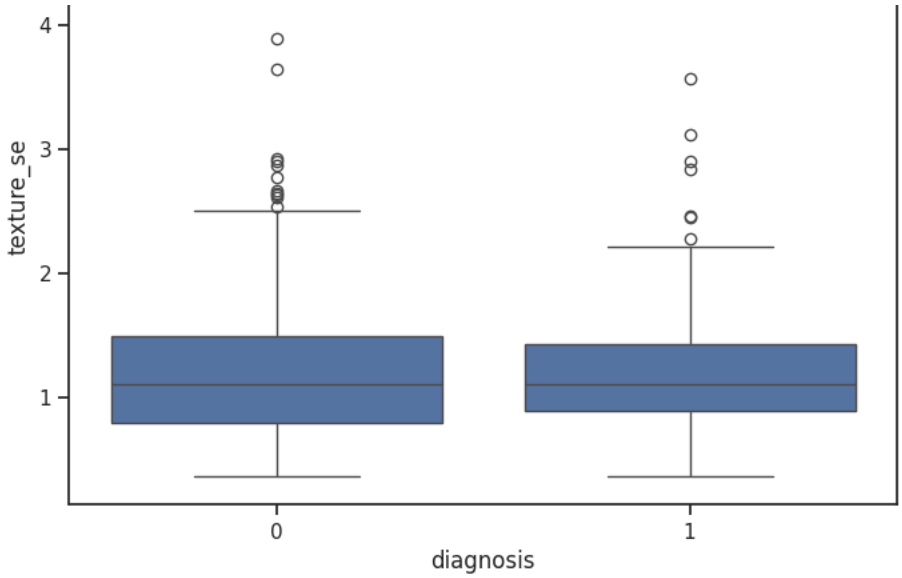

diagnosis vs symmetry_mean
0.200
0.225
0.250
0.275
0.300

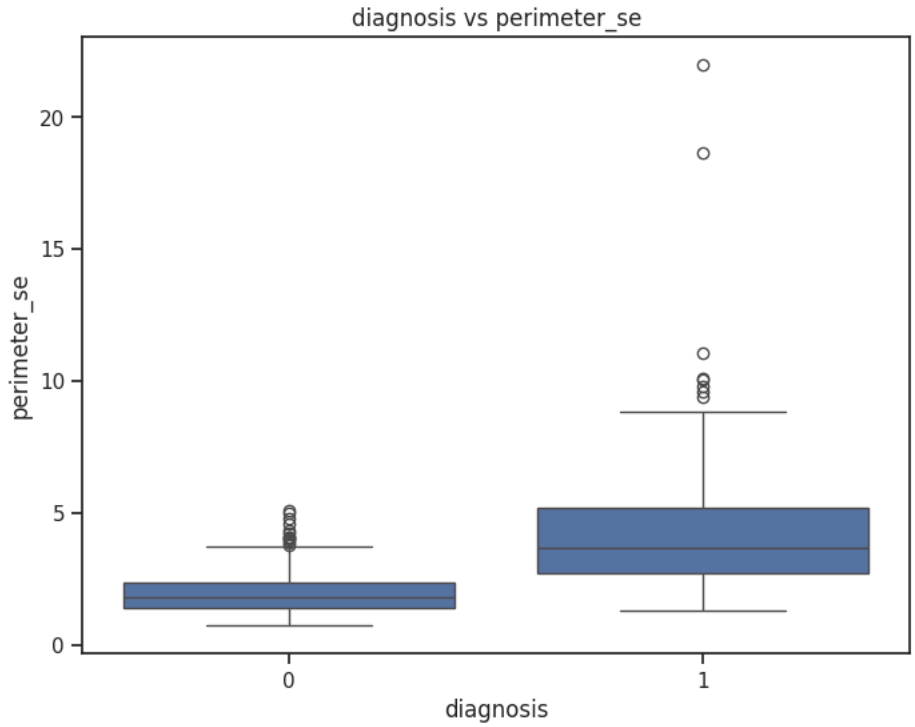

sy
0.175
0.150
0.125
0.100
0
1
diagnosis

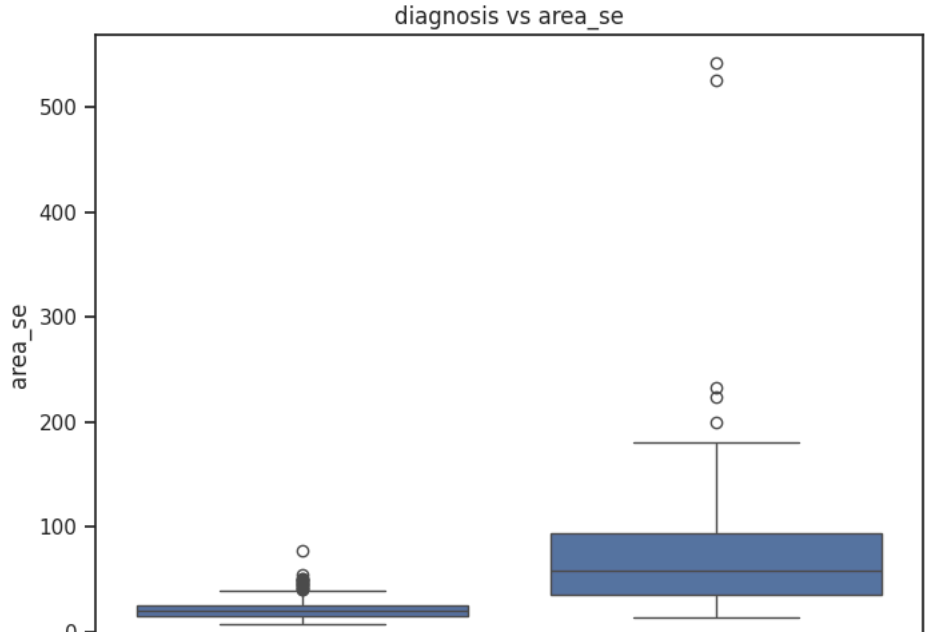

diagnosis vs fractal_dimension_mean
fractal_dimension_mean
0.09
0.08
0.07
0.06
0.05
0
1
diagnosis

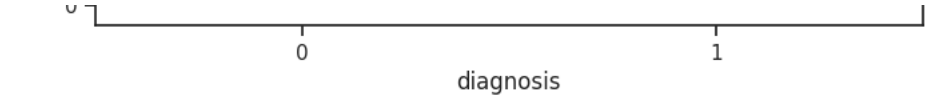

diagnosis vs radius_se
radius_se
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
diagnosis

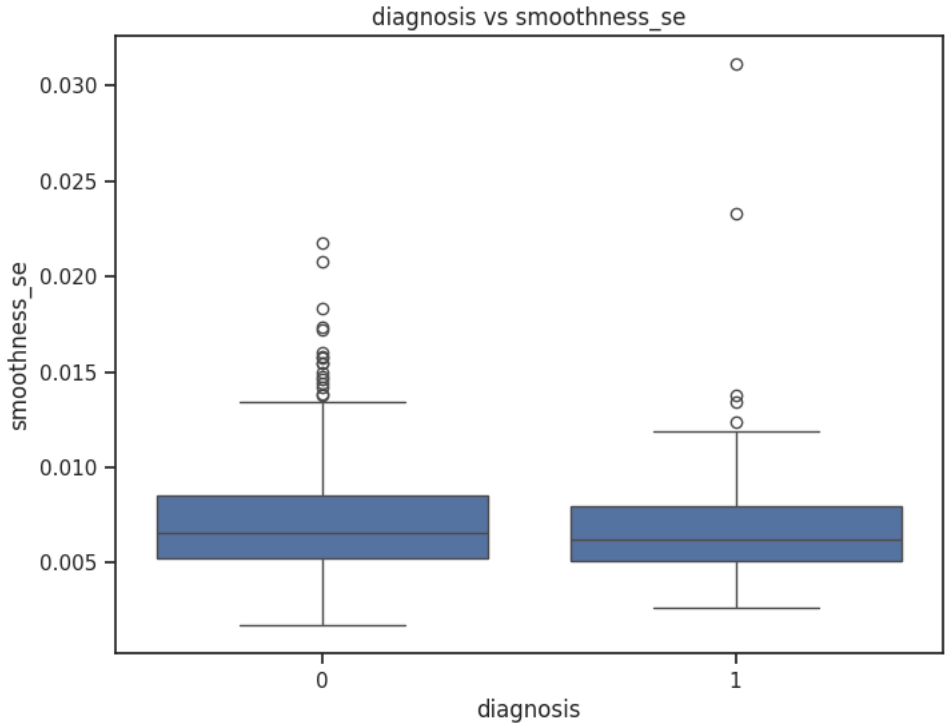

diagnosis vs texture_se
5

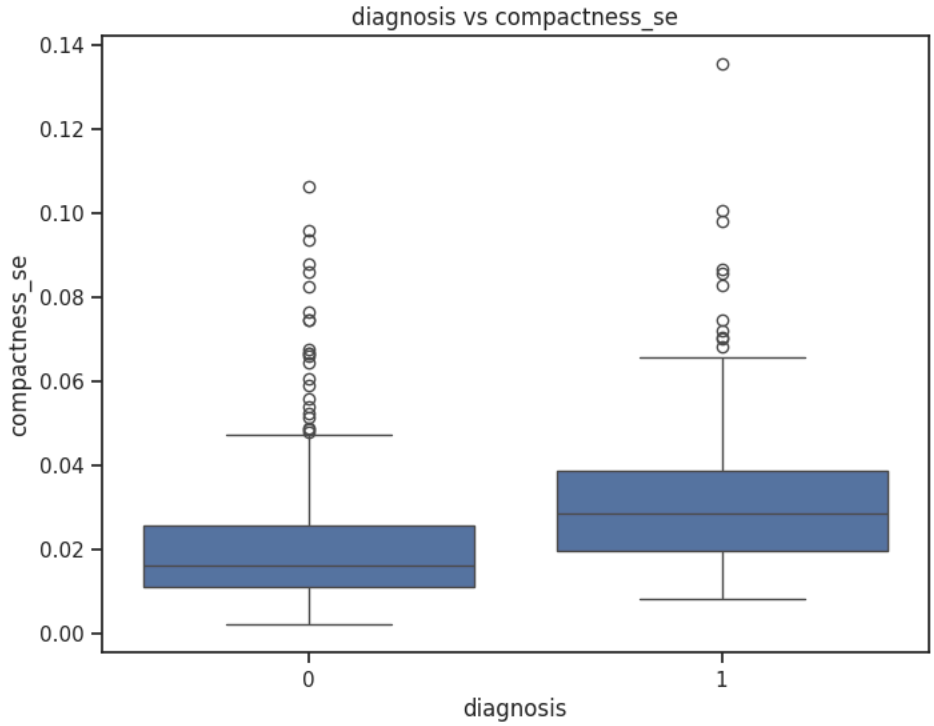

texture_se
4
3
2
1
0
1
diagnosis

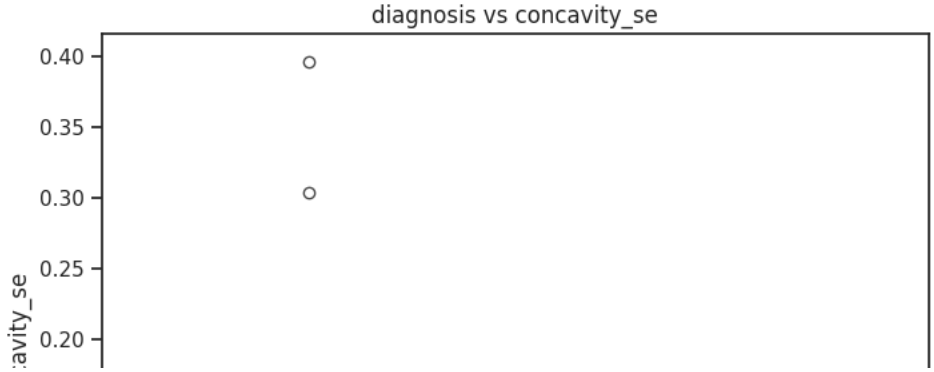

diagnosis vs perimeter_se
perimeter_se
20
15
10
5
0
0
1
diagnosis

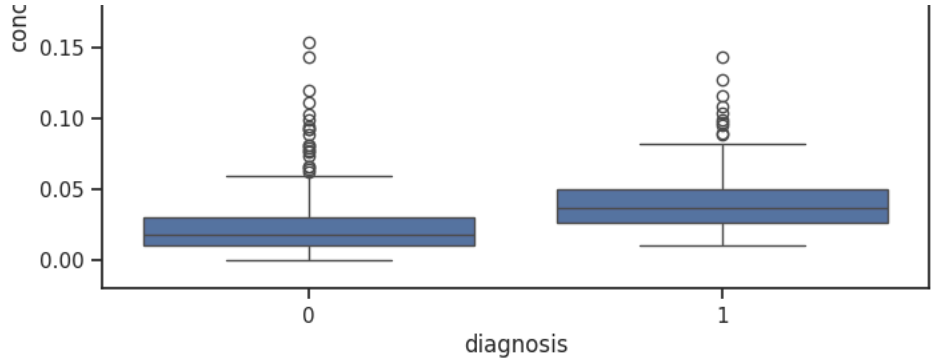

diagnosis vs area_se
area_se
500
400
300
200
100

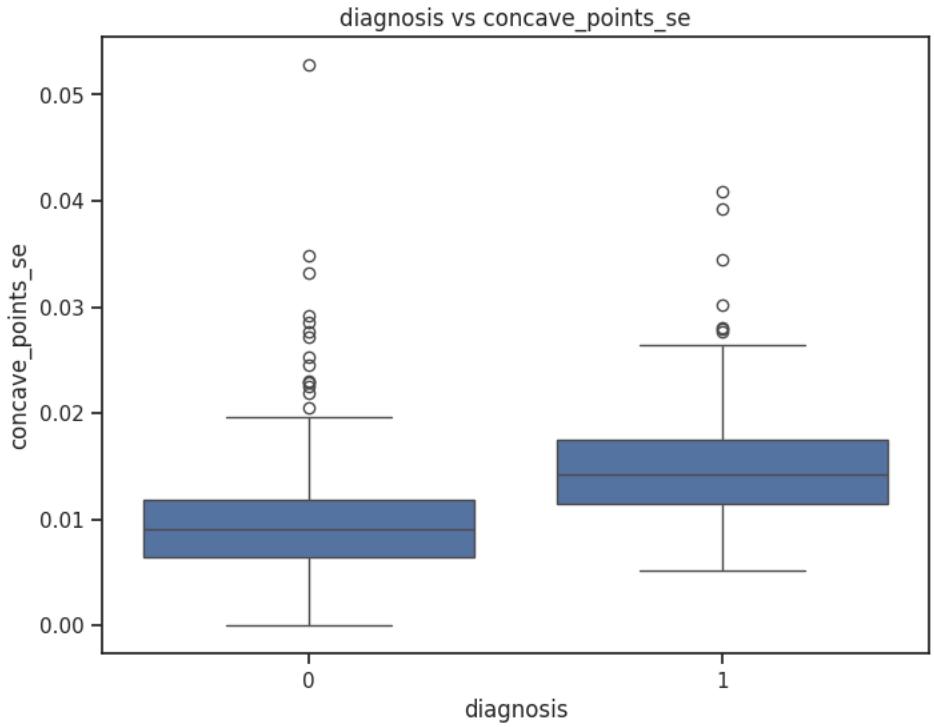

0
1
diagnosis

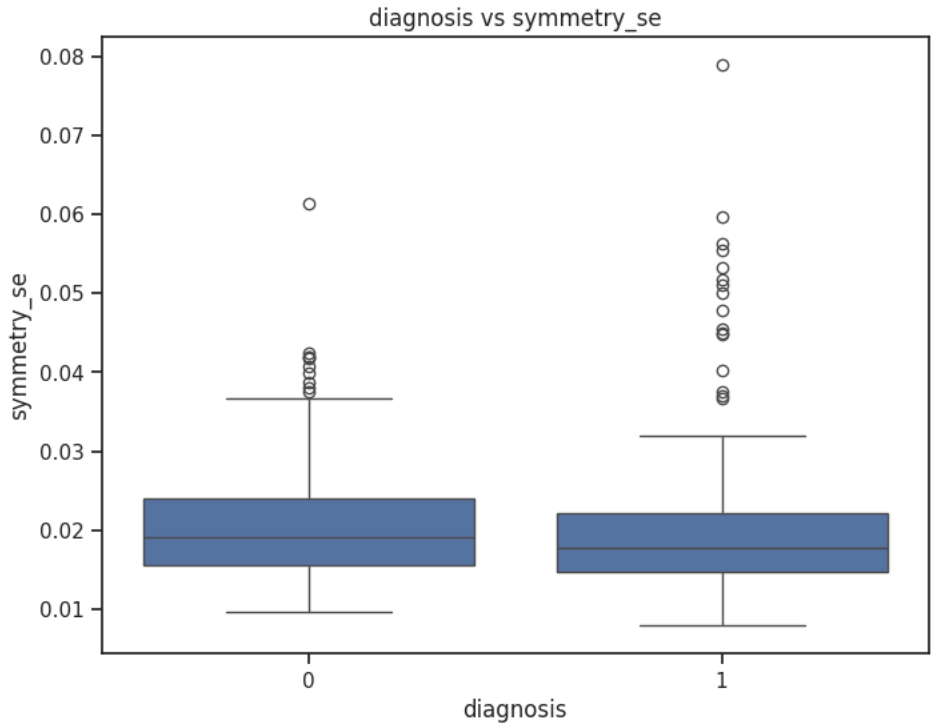

diagnosis vs smoothness_se
0.030
0.025
0.020
0.015
0.010
0.005
smoothness_se
0
1
diagnosis

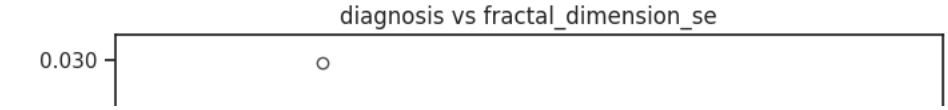

diagnosis vs compactness_se
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
compactness_se
0
1
diagnosis

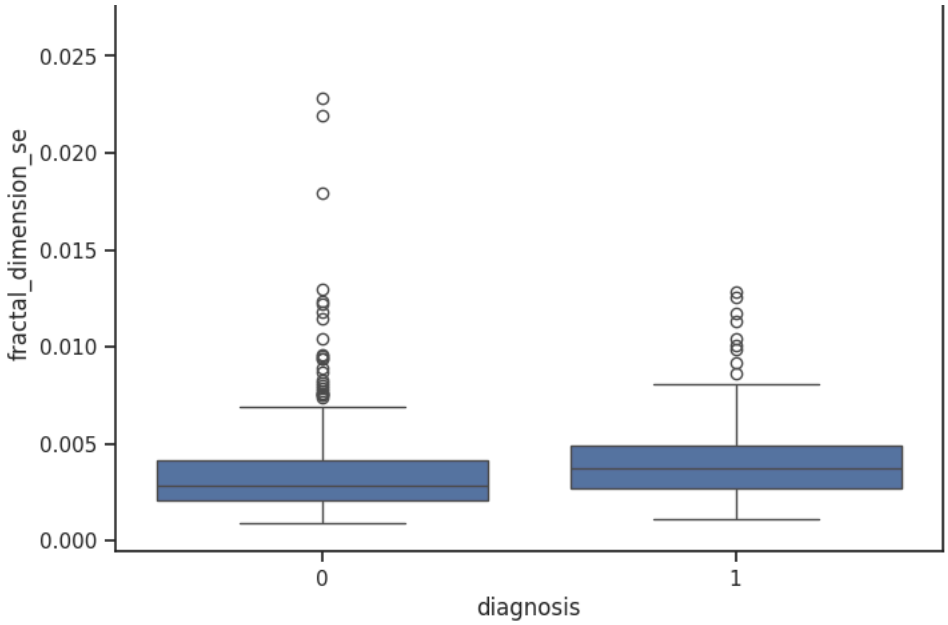

diagnosis vs concavity_se
0.40
0.35
0.30
0.25
0.20

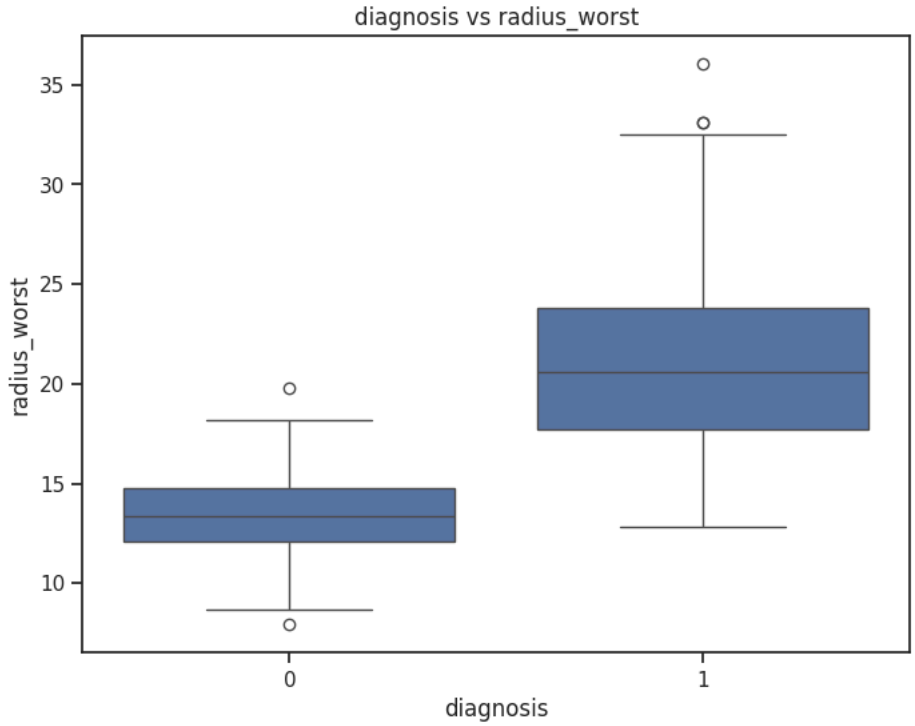

conc
0.15
0.10
0.05
0.00
0
1
diagnosis

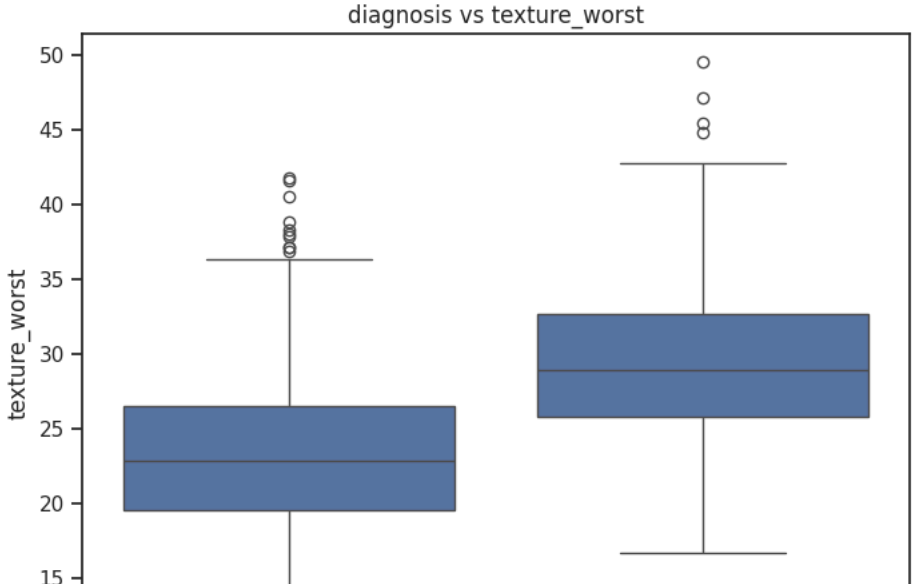

diagnosis vs concave_points_se
concave_points_se
0.05
0.04
0.03
0.02
0.01
0.00
0
1
diagnosis

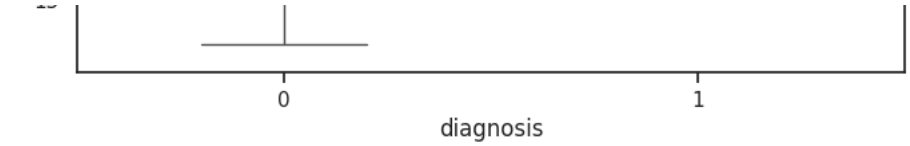

diagnosis vs symmetry_se
symmetry_se
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0
1
diagnosis

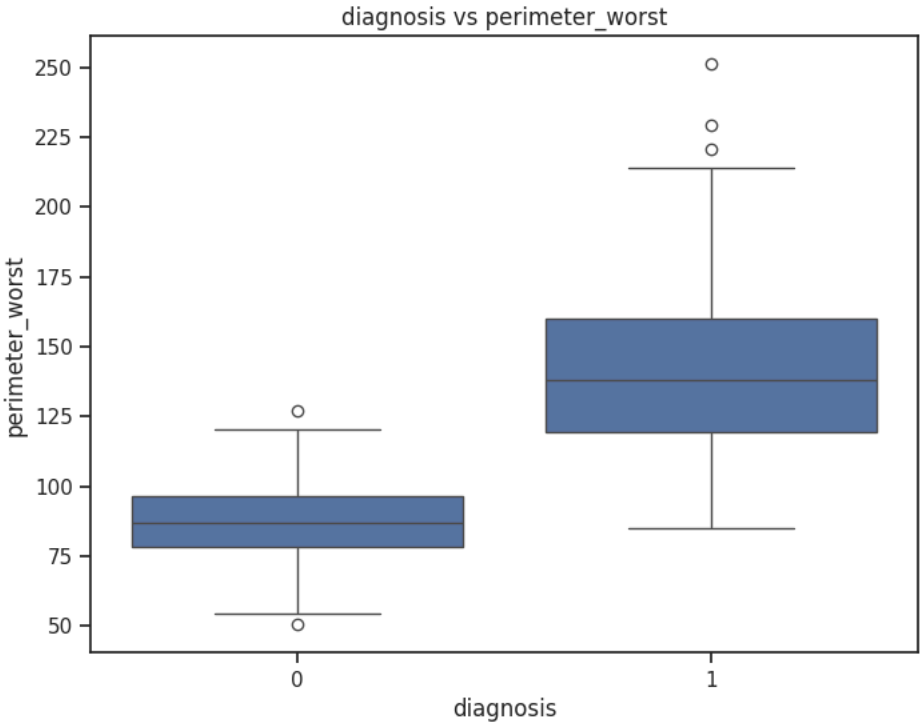

diagnosis vs fractal_dimension_se
0.030

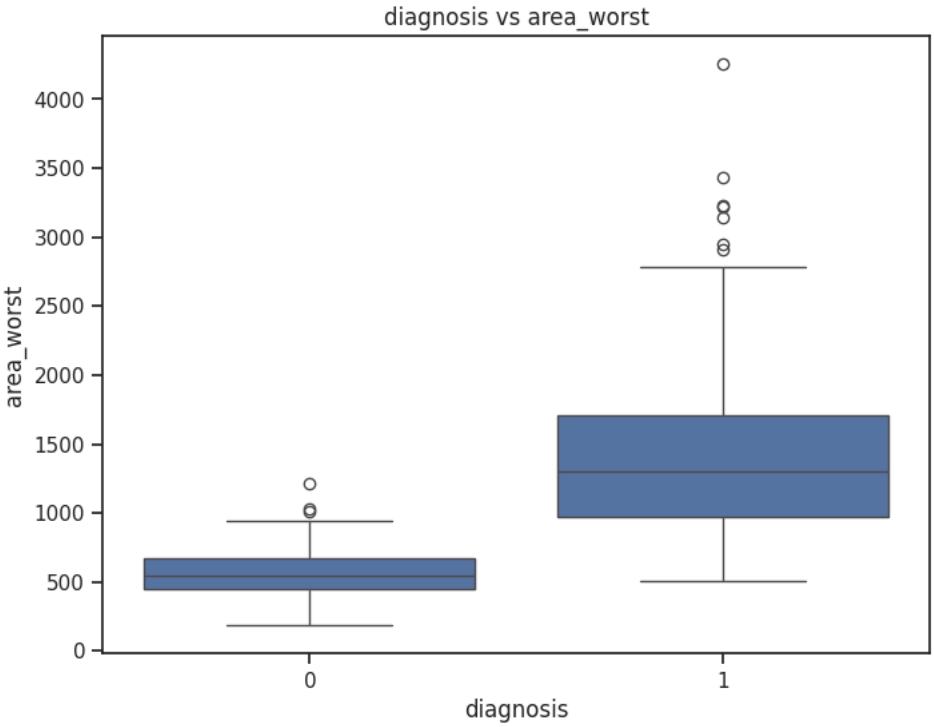

fractal_dimension_se
0.025
0.020
0.015
0.010
0.005
0.000
0
1
diagnosis

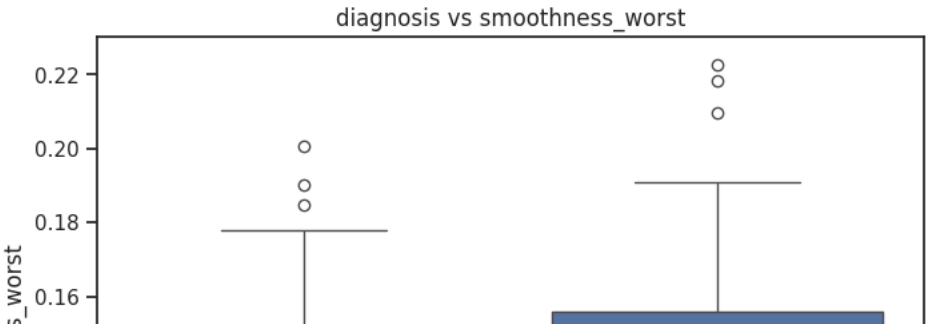

diagnosis vs radius_worst
radius_worst
35
30
25
20
15
10
0
1
diagnosis

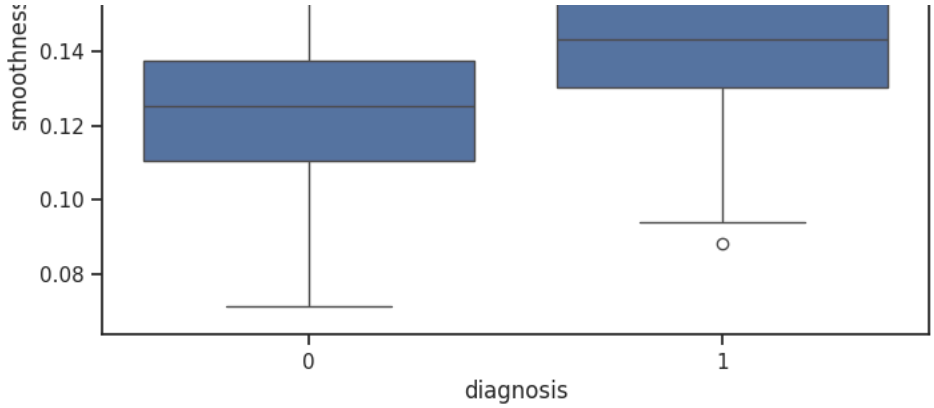

diagnosis vs texture_worst
texture_worst
50
45
40
35
30
25
20
15

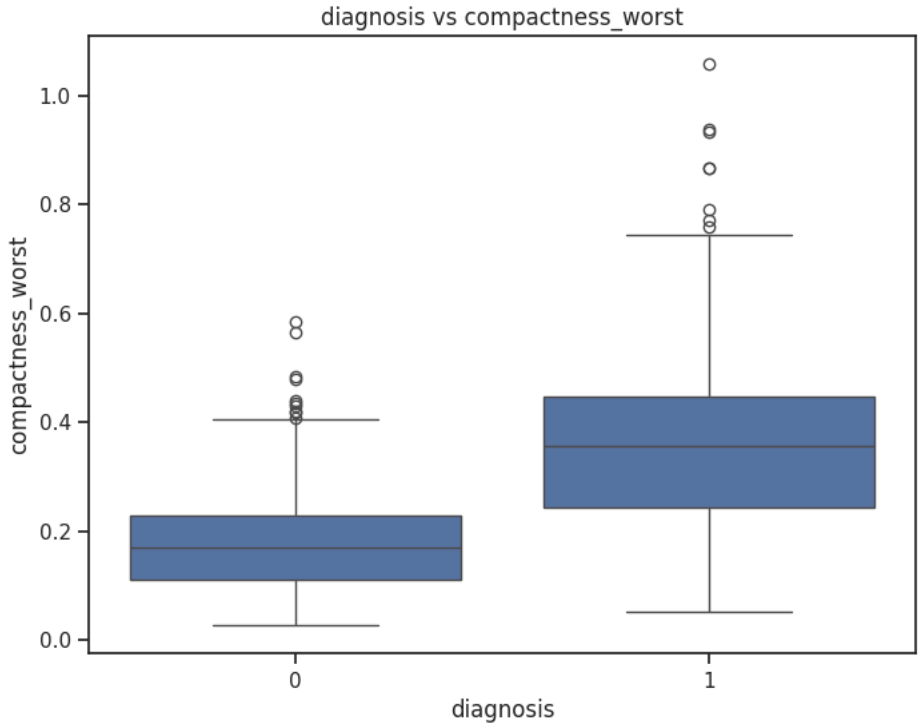

0
1
diagnosis

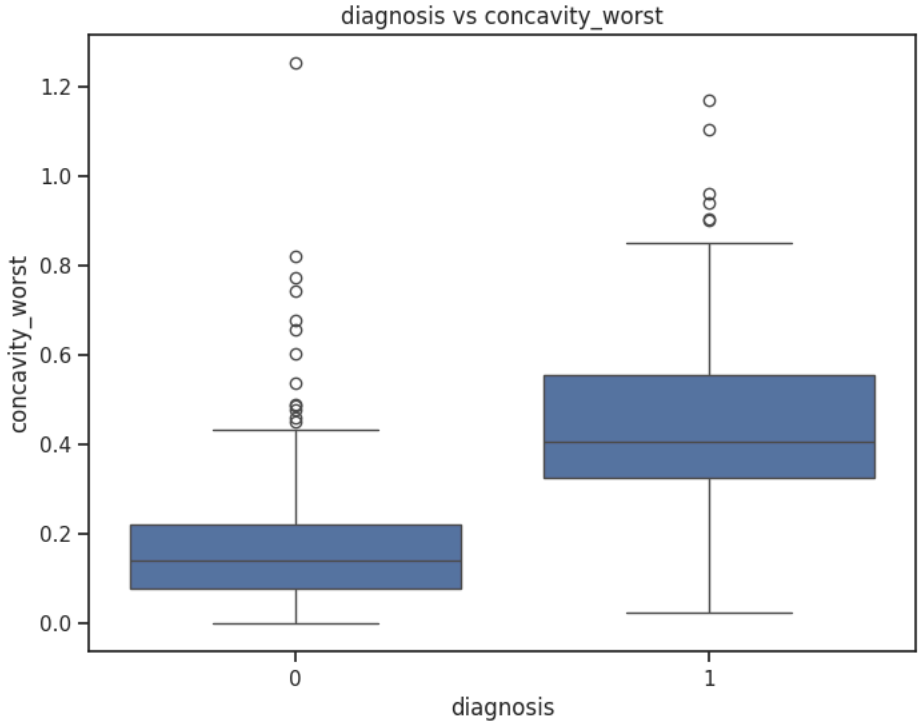

diagnosis vs perimeter_worst
perimeter_worst
250
225
200
175
150
125
100
75
50
0
1
diagnosis

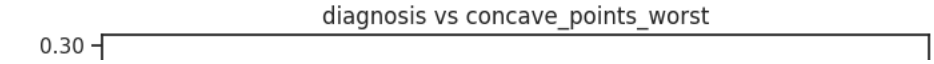

diagnosis vs area_worst
area_worst
4000
3500
3000
2500
2000
1500
1000
500
0
0
1
diagnosis

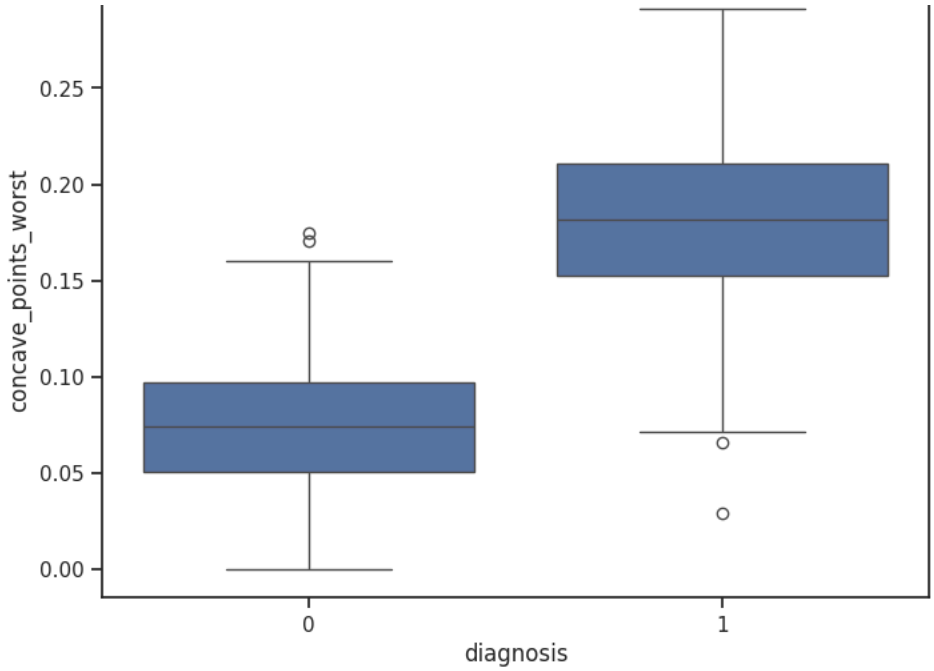

diagnosis vs smoothness_worst
0.22
0.20
0.18
0.16

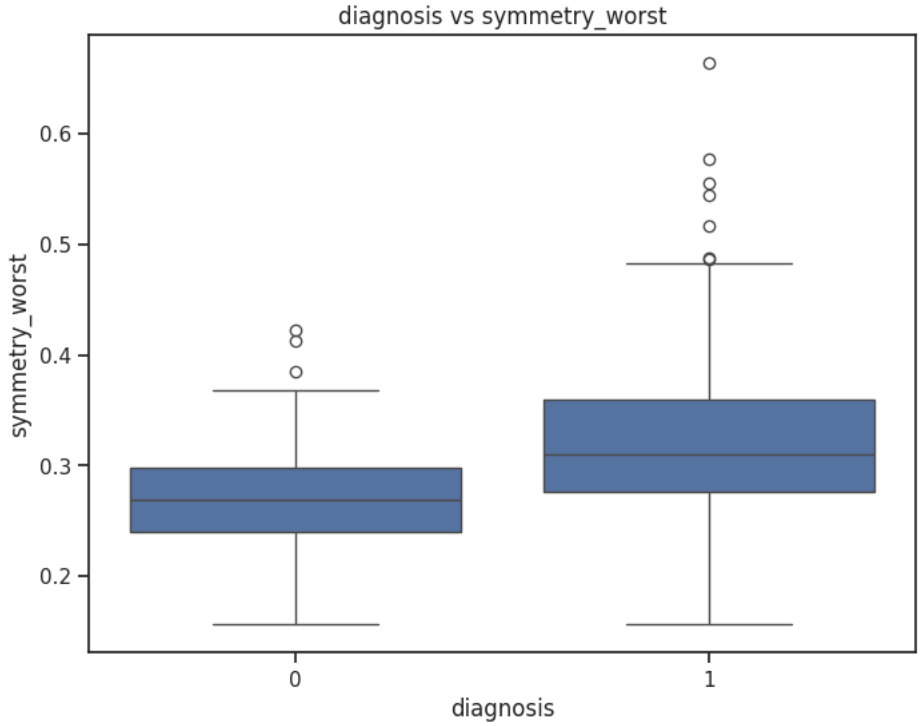

smoothness
0.14
0.12
0.10
0.08
0
1
diagnosis

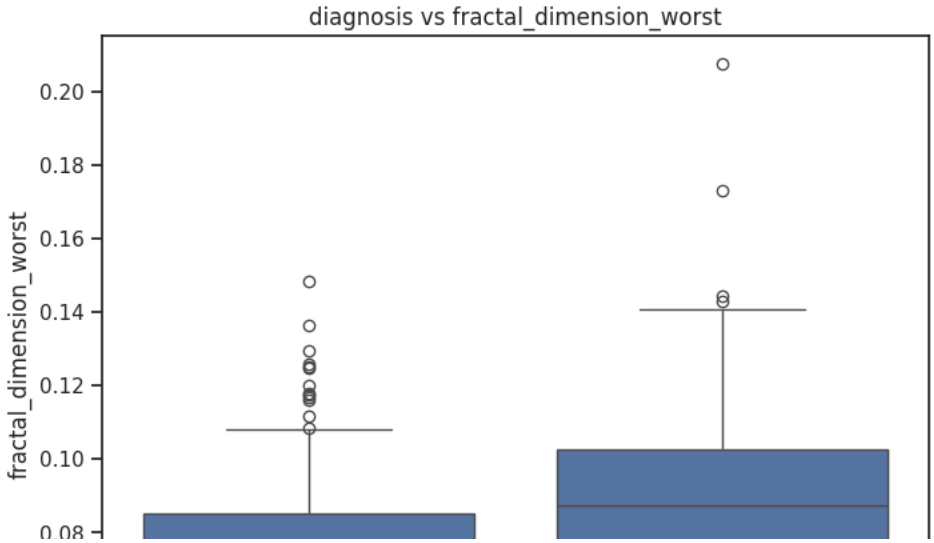

diagnosis vs compactness_worst
compactness_worst
1.0
0.8
0.6
0.4
0.2
0.0
0
1
diagnosis

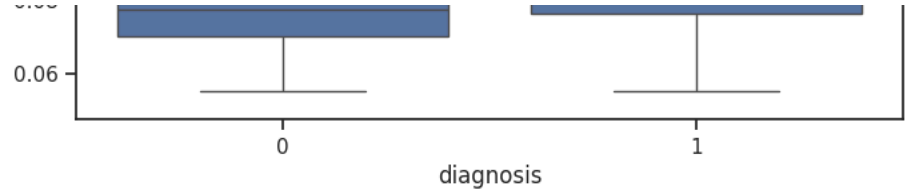

diagnosis vs concavity_worst
concavity_worst
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
1
diagnosis


diagnosis vs concave_points_worst
0.30


concave_points_worst
0.25
0.20
0.15
0.10
0.05
0.00
0
1
diagnosis


diagnosis vs symmetry_worst
symmetry_worst
0.6
0.5
0.4
0.3
0.2
0
1
diagnosis


diagnosis vs fractal_dimension_worst
fractal_dimension_worst
0.20
0.18
0.16
0.14
0.12
0.10
0.08



From the above graph, we can see how the affected rate is ditributed and for each of the columns which means we can observe that the how the level of each of the columns are for the two categories of the diseases and how they are distributed.

In the below code, we are removing the unwanted column id which is not necessary for the further analysis.

```
# Drop non-required columns ('id' column)
df.drop(columns=['id'], inplace=True)
```

In the below code, we are seperating the independent and dependent variables and storing it in variables X and Y.

```
# Separate the features (X) and the target variable (y)
X = df.drop(columns=['diagnosis'])  # Features
y = df['diagnosis']  # Target variable
```

In the below code, we are importing the necessary library for standardizing the variables.

```
from sklearn.preprocessing import StandardScaler
```

In the below cide, we are standardizing the X (independent variable).

```
# Perform standardization on the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In the below code, we are importing the necessary library to split the dataset for training and testing.

```
from sklearn.model_selection import train_test_split
```

In the below code, we are spliting the dataset for training and testing (both independent and dependent) and storing them in the variable.

```

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

In the below code, we are imprting the necessary packkages to perform LDA and logistic regression and futher analysis.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

In the below code, we are training the model using taring data for LDA

```
# 11. Implement Linear Discriminant Analysis (LDA)
# a. Train the LDA model using training data
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
```

```
▾ LinearDiscriminantAnalysis
LinearDiscriminantAnalysis()
```

In the below code, we are evealuating the performance of the trained model by predicting the values for trained independent variable and untarined independent varibale which basically the independent variable from the training dataset and the testing dataset.

```
# b. Evaluate the performance of the trained model using appropriate metrics
lda_train_preds = lda.predict(X_train)
lda_test_preds = lda.predict(X_test)
```

In the below code, we are checking the accuracy score of the model both for trained dataset and the testing dataset.

```
lda_train_accuracy = accuracy_score(y_train, lda_train_preds)
lda_test_accuracy = accuracy_score(y_test, lda_test_preds)
```

In the below code, we printing the accuracy score of the model for both the training and the testing dataset.

```
print("Linear Discriminant Analysis (LDA) Model Performance:")
print("Train Accuracy:", lda_train_accuracy)
print("Test Accuracy:", lda_test_accuracy)
```

```
Linear Discriminant Analysis (LDA) Model Performance:
Train Accuracy: 0.9648351648351648
Test Accuracy: 0.956140350877193
```

From the above result we can infer that the,

- Accuracy score of model for Training Dataset is 96.5%
- Accuracy score of model for Testing Dataset is 95.6%

In the below code, we building the Logistic regression model and we are training the model with training dataset.

```
# 12. Implement Logistic Regression
# a. Train the Logistic Regression model using the training data
logreg = LogisticRegression(max_iter=1000)  # Increase max_iter if convergence warnings occur
logreg.fit(X_train, y_train)

```

```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

In the below code, we are evaluating the performance of the trained model both for trained dataset and the testing dataset.

```
# b. Evaluate the performance of the trained model using appropriate metrics
logreg_train_preds = logreg.predict(X_train)
logreg_test_preds = logreg.predict(X_test)
```

In the below code, we are getting the accuracy score for the model for both trained dataset and the testing dataset.

```
logreg_train_accuracy = accuracy_score(y_train, logreg_train_preds)
logreg_test_accuracy = accuracy_score(y_test, logreg_test_preds)
```

In the below code, we are printing the Accuracy score for the LR model for both training and testing dataset.

```
print("\nLogistic Regression Model Performance:")
print("Train Accuracy:", logreg_train_accuracy)
print("Test Accuracy:", logreg_test_accuracy)
```

```
Logistic Regression Model Performance:
Train Accuracy: 0.9868131868131869
Test Accuracy: 0.9736842105263158
```

From the above result, we can observe that the the,

- Accurcay score of model for training dataset is 98.6%
- Accuracy score of model for testing dataset is 97.4%

The below code allows for a concise summary of the performance of the LDA model in terms of precision, recall, and the confusion matrix, which helps in evaluating the model's effectiveness in predicting the target variable.

```
# 13. Display the classification report and confusion matrix
print("\nClassification Report and Confusion Matrix for Linear Discriminant Analysis:")
print(classification_report(y_test, lda_test_preds))
print("Confusion Matrix:")
print(confusion_matrix(y_test, lda_test_preds))
```

```
Classification Report and Confusion Matrix for Linear Discriminant Analysis:
              precision    recall  f1-score   support

           0       0.95      0.99      0.97        71
           1       0.97      0.91      0.94        43

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114

Confusion Matrix:
[[70  1]
 [ 4 39]]
```

The provided output includes the classification report and confusion matrix for the results of a Linear Discriminant Analysis (LDA) model.

- Classification Report:
  - The classification report provides a summary of performance metrics such as precision, recall, F1-score, and support for each class (in this case, classes 0 and 1).
    - For class 0 (presumably representing benign tumors):
      - Precision: 0.95 (i.e., when the model predicts a tumor as benign, it is correct about 95% of the time)
      - Recall: 0.99 (i.e., the model correctly identifies about 99% of actual benign tumors)
      - F1-score: 0.97 (the harmonic mean of precision and recall, providing a balance between them)
      - Support: 71 (the number of samples in the test set with actual class 0)
    - For class 1 (presumably representing malignant tumors):
      - Precision: 0.97 (i.e., when the model predicts a tumor as malignant, it is correct about 97% of the time)
      - Recall: 0.91 (i.e., the model correctly identifies about 91% of actual malignant tumors)
      - F1-score: 0.94
      - Support: 43 (the number of samples in the test set with actual class 1)
      - Accuracy: 0.96 (overall accuracy of the model in predicting both classes)

Macro average and weighted average metrics provide an average of precision, recall, and F1-score across all classes, considering class imbalance. Confusion Matrix:

- The confusion matrix summarizes the performance of the classification model by showing the count of true positive, true negative, false positive, and false negative predictions.

In the confusion matrix:

- The top-left cell (70) represents true negatives (TN) - the number of correctly predicted benign tumors.
- The top-right cell (1) represents false positives (FP) - the number of benign tumors incorrectly predicted as malignant.
- The bottom-left cell (4) represents false negatives (FN) - the number of malignant tumors incorrectly predicted as benign.
- The bottom-right cell (39) represents true positives (TP) - the number of correctly predicted malignant tumors.

This information provides insights into the model's performance, indicating its ability to correctly classify tumors as benign or malignant, as well as any potential errors in prediction.

The below code allows for the evaluation of the Logistic Regression model's performance in terms of precision, recall, F1-score, and confusion matrix, providing insights into its effectiveness in predicting the target variable.

```
print("\nClassification Report and Confusion Matrix for Logistic Regression:")
print(classification_report(y_test, logreg_test_preds))
print("Confusion Matrix:")
print(confusion_matrix(y_test, logreg_test_preds))
```

```
Classification Report and Confusion Matrix for Logistic Regression:
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        71
           1       0.98      0.95      0.96        43

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

Confusion Matrix:
[[70  1]
 [ 2 41]]
```

The provided output includes the classification report and confusion matrix for the results of a Logistic Regression model.

- Classification Report:
  - Precision:
    - For class 0 (presumably representing benign tumors), precision is 0.97, indicating that when the model predicts a tumor as benign, it is correct about 97% of the time.
    - For class 1 (presumably representing malignant tumors), precision is 0.98, meaning that when the model predicts a tumor as malignant, it is correct about 98% of the time.
  - Recall:
    - For class 0, recall is 0.99, suggesting that the model correctly identifies about 99% of actual benign tumors.
    - For class 1, recall is 0.95, indicating that the model correctly identifies about 95% of actual malignant tumors.
  - F1-score:
    - The harmonic mean of precision and recall.
    - For class 0, it is 0.98.
    - For class 1, it is 0.96.
  - Support:
    - The number of samples in the test set with actual class 0 is 71, and for class 1, it is 43.
  - Accuracy: Overall accuracy of the model in predicting both classes is 0.97.
- Confusion Matrix:
  - The confusion matrix summarizes the performance of the classification model by showing the count of true positive, true negative, false positive, and false negative predictions.
  - In the confusion matrix, the top-left cell (70) represents true negatives (TN) - the number of correctly predicted benign tumors.
  - The top-right cell (1) represents false positives (FP) - the number of benign tumors incorrectly predicted as malignant.
  - The bottom-left cell (2) represents false negatives (FN) - the number of malignant tumors incorrectly predicted as benign.
  - The bottom-right cell (41) represents true positives (TP) - the number of correctly predicted malignant tumors.

The below code allows for the visualization and comparison of the confusion matrices, providing insights into the performance of the classification models in predicting the target variable.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot confusion matrices for both LDA and Logistic Regression models
plt.figure(figsize=(12, 6))

# Confusion matrix for LDA
plt.subplot(1, 2, 1)
lda_cm = confusion_matrix(y_test, lda_test_preds)
sns.heatmap(lda_cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix - LDA')
plt.xlabel('Predicted')
plt.ylabel('Actual')

# Confusion matrix for Logistic Regression
plt.subplot(1, 2, 2)
logreg_cm = confusion_matrix(y_test, logreg_test_preds)
sns.heatmap(logreg_cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.tight_layout()
plt.show()
```

Insights from the graph:

- Both models perform similarly in correctly predicting class 0, with 70 TN each.
- Logistic Regression performs slightly better in predicting class 1, with 41 TP compared to LDA's 39 TP.
- Logistic Regression also has fewer FN, with only 2 compared to LDA's 4, indicating it is better at identifying class 1 instances.
- Both models have a low FP rate, with only 1 FP each, indicating that they both are good at not misclassifying class 0 instances as class 1.

Overall, the Logistic Regression model appears to have a slight edge over the LDA model in terms of overall accuracy and precision in this particular dataset.

```
The below code allows for a visual comparison of the accuracies of the
two models, providing insights into their relative performance.
```

The below code allows for a visual comparison of the accuracies of the two models, providing insights into their relative performance.

```
--INSERT--
```

```
# Compare accuracies of both models using a bar plot
models = ['LDA', 'Logistic Regression']
accuracies = [lda_test_accuracy, logreg_test_accuracy]

plt.figure(figsize=(8, 5))
sns.barplot(x=models, y=accuracies, palette='Blues')
plt.title('Comparison of Model Accuracies')
plt.xlabel('Models')
plt.ylabel('Accuracy')
plt.ylim(0, 1)  # Set y-axis limit to ensure visibility of differences
plt.show()
```

```
<ipython-input-34-e02f553c89df>:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legen

  sns.barplot(x=models, y=accuracies, palette='Blues')
```